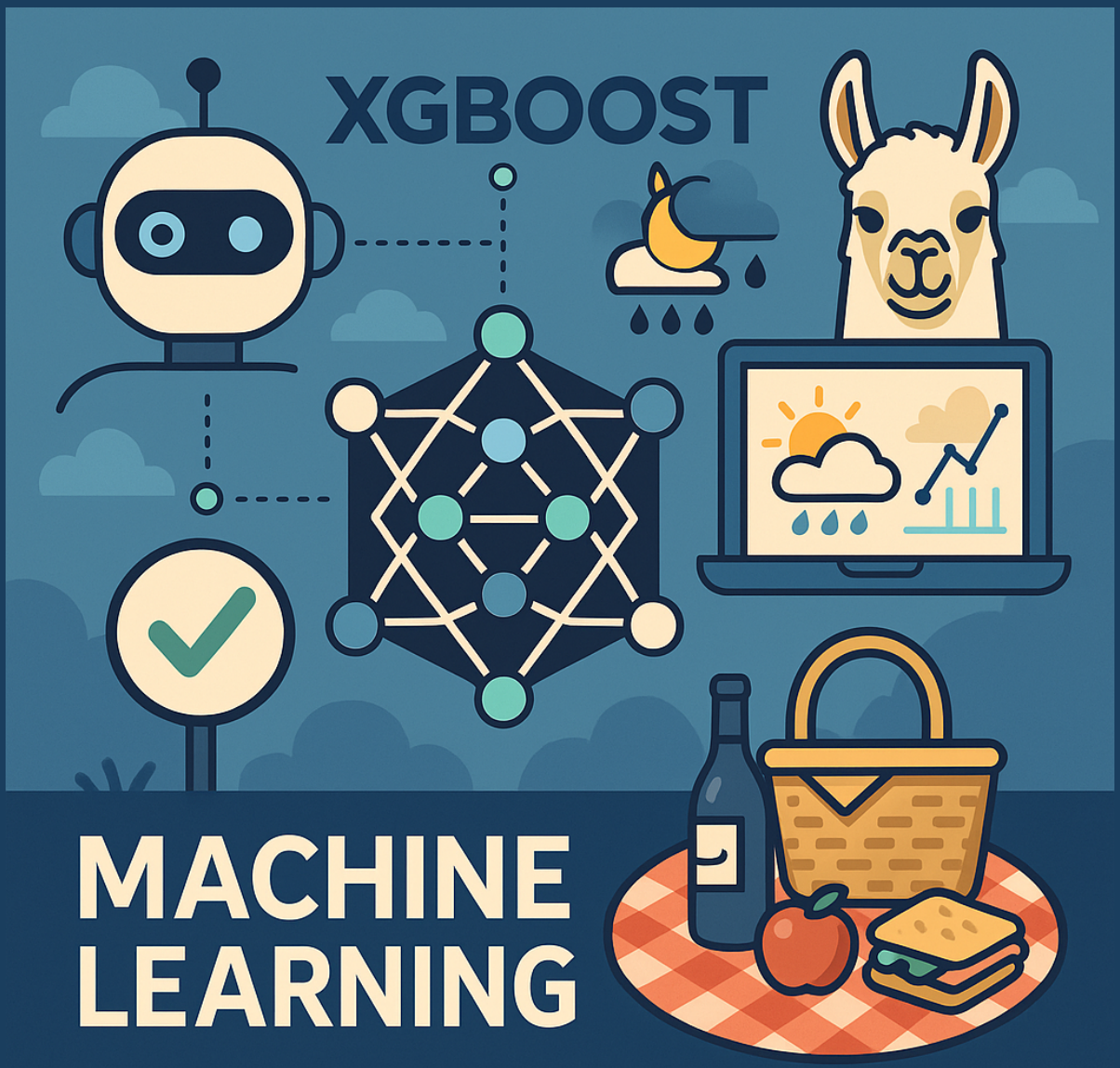
Will it Picnic?
Silicon Valley Meets Strawberry Jam

Alessandra Balestrieri, Antonio Carboncino, Libera Pasqualina Ciambriello

Algoritmi e Applicazioni per l'Intelligenza Artificiale

Luglio 2025



Contents

1 Introduction **3**

2 About the data **5**

2.1 Feature Analysis 7

2.2 Label Analysis 9

3 Data Cleaning **11**

3.1 Some issues 11

3.2 What is data cleaning? 11

3.3 Handling Missing Data 12

3.4 Treating Outliers 13

3.5 Handling Categorical Features 16

3.6 Standardizing Data 17

4 Feature selection **19**

4.1 Pearson's Correlation 20

5 Dimensionality reduction **23**

5.1 PCA 23

5.2 t-SNE 30

6 Classification **33**

6.1 Decision Tree 35

6.2 Random forest 44

6.3 XGBoost 48

6.4 K-NN 51

6.5 Deep Learning 56

7 Conclusion **71**

Chapter 1

Introduction

Weather forecasting plays a vital role in everyday life, influencing sectors such as agriculture, transportation, and emergency management. Although conventional forecasting techniques have advanced considerably, there remains significant potential for gains in both accuracy and efficiency.

In recent years, machine learning (ML) has emerged as a powerful tool for weather prediction: by harnessing the vast streams of data collected from global weather stations, ML models can uncover complex patterns and subtle relationships, delivering more precise and timely forecasts.



In this study, we investigate the application of supervised learning algorithms for predicting atmospheric conditions. We employ a dataset of daily observations from multiple weather stations within a defined region, using historical measurements to train our models and generate out-of-sample forecasts.

Our research addresses questions like "*Which ML algorithms are best suited for weather forecasting tasks?*", "*Which features most strongly drive model performance?*" By systematically evaluating these factors, we aim to establish best practices for deploying machine-learning techniques in real-world weather prediction.

We begin by analysing the **nature of our data**: daily measurements from a network of weather stations, each variable revealing its own subtleties and challenges. **Cleaning** and preparing these measurements becomes the focus of our

next discussion, where we tackle missing values and outliers, translate categorical readings into numeric forms, and standardize everything for fair treatment by our algorithms.

Once the dataset is robust, we explore which pieces of information truly matter. Through **correlation analyses** and dimensionality-reduction techniques like **PCA** and **t-SNE**, we peel back layers of complexity to expose the variables that most strongly influence weather outcomes. This sets the stage for the heart of our work: training and comparing a suite of supervised models, such as **Decision Trees** and **k-NN**. We complement these with **Neural Networks** to see how far predictive power can be pushed.

Finally, we put together all results together in a reflective conclusion, highlighting which algorithms delivered the most reliable forecasts.

Chapter 2

About the data

The initial meteorological data were obtained from the ECA&D project (European Climate Assessement & Dataset), which provides daily observations from meteorological stations across Europe and the Mediterranean. It has been selected 18 locations for which continuous daily records were available from 2000 through 2010.

The 18 locations included in this study are:



- Basel (Switzerland)
- Budapest (Hungary)
- Dresden, Düsseldorf, Kassel, Munich (Germany)
- De Bilt, Maastricht (Netherlands)
- Heathrow (United Kingdom)
- Ljubljana (Slovenia)
- Malmö, Stockholm (Sweden)
- Montélimar, Perpignan, Tours (France)
- Oslo (Norway)
- Rome (Italy)
- Sonnblick (Austria)

Although historical records for some stations extend back into the 19th century, here we restrict our analysis to the period 2000–2010. This 11-year span (including leap years) yields 3654 daily observations for each location, forming the combined dataset across all 18 sites, which specifies the day and month in which each recording is made.

For each station, we retained the temperature variables

- Mean Temperature
- Maximum Temperature
- Minimum Temperature

and in addition, where available, we included:

- Cloud cover
- Wind speed
- Wind gust
- Humidity
- Sea-level pressure
- Global radiation
- Precipitation
- Sunshine duration

2.1 Feature Analysis

To analyse the dataset, the first step is to separate the features (input variables) from the target (label). A feature is an individual characteristic or attribute of the data being used in a machine learning model, while a label is the target attribute that the machine learning model is trying to predict.

In our case, features can be divided into numerical and categorical types, more specifically, binary features.

Numeric features are continuous or discrete numerical values. These features can be used directly in machine learning models.

Categorical features are values that represent categories or classes. These features must be transformed into numerical values before they can be used in machine learning models.

Binary features can be considered as a subset of categorical features, since they represent categories with only two possible values, often represented as 1/0 or yes/no.

Let's describe the two CSV files in our weather dataset.

The dataset includes two temporal features. The first, `_DATE`, is an integer column representing the date of observation, encoded in the `YYYYMMDD` format (e.g., 20000101 corresponds to January 1, 2000). The second, `_MONTH`, is also stored as an integer and indicates the calendar month extracted from the `_DATE` column, with values ranging from 1 (January) to 12 (December).

We chose not to include the complete `_DATE` column in our study because it is almost always useless or harmful (it introduces noise since each day is unique), so it provides no meaningful information.

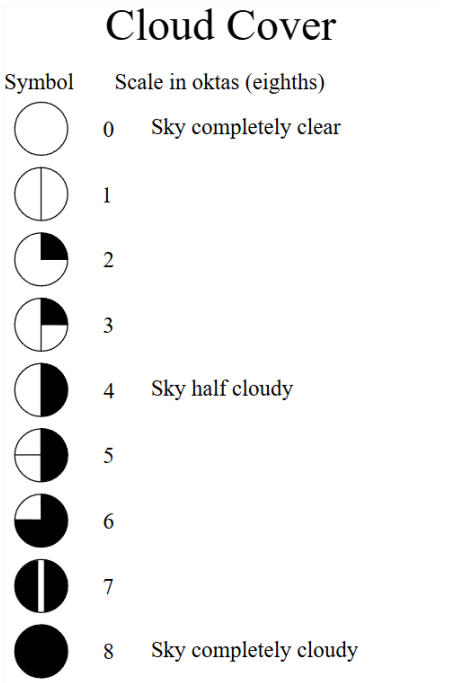
Instead, we consider the `_MONTH` column because interesting patterns might emerge from which significant seasonal effects could be deduced.

For each of the 18 weather stations, the following daily weather measurements appear as separate columns. Below we show a table describing the type and the unit of measurement of each feature.

Suffix	Description	Units	Type
`_MONTH`	Calendar month	-	Categorical
`_temp_mean`	Mean temperature	in 1°C	Numeric
`_temp_min`	Minimum temperature	in 1°C	Numeric
`_temp_max`	Maximum temperature	in 1°C	Numeric
`_cloud_cover`	Cloud cover	oktas	Categorical
`_humidity`	Relative humidity	in 1%	Numeric
`_pressure`	Sea level pressure	in 1000 hPa	Numeric
`_global_radiation`	Global solar radiation	in 100 W/m^2	Numeric
`_precipitation`	Daily precipitation	in 10 mm	Numeric
`_sunshine`	Hours of sunshine	in 0.1 hours	Numeric
`_wind_speed`	Wind speed	in 1 m/s	Numeric
`_wind_gust`	Wind gust speed	in 1 m/s	Numeric
`_picnic_label`	Label class	-	Categorial

All features are numeric, except for the `_cloud_cover`, `_MONTH` and the `_picnic_label`.

`_cloud_cover` is provided in *oktas*. In meteorology, an okta is a scale of measurement used to describe the amount of cloud cover at any given location such as a weather station. Sky conditions are estimated in terms of how many eighths of the sky are covered in cloud, ranging from 0 oktas (completely clear sky) through to 8 oktas (completely overcast).



2.2 Label Analysis

The file `weather_prediction_picnic_labels.csv` contains one Boolean column per city:

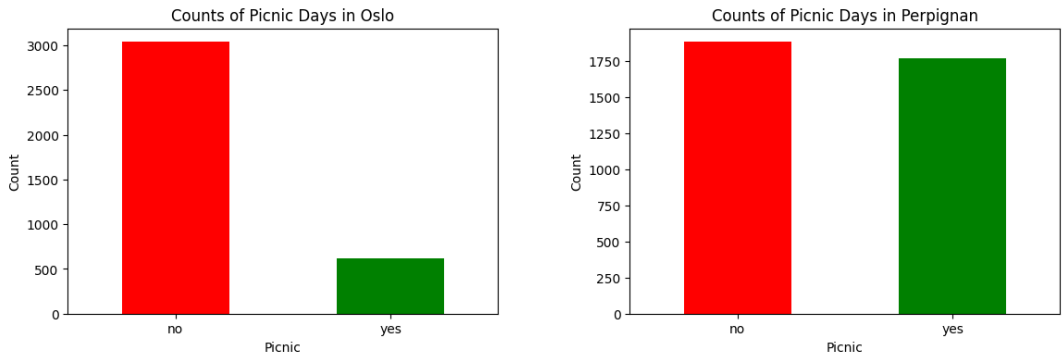
`<CITY>_picnic_weather` (`bool`): `True` if, on that date at that location, conditions were judged suitable for a picnic; `False` otherwise.

There is no single unified "label" column. Instead, we can frame our problem per city, using each `<CITY>_picnic_weather` as the target variable for its corresponding weather features.

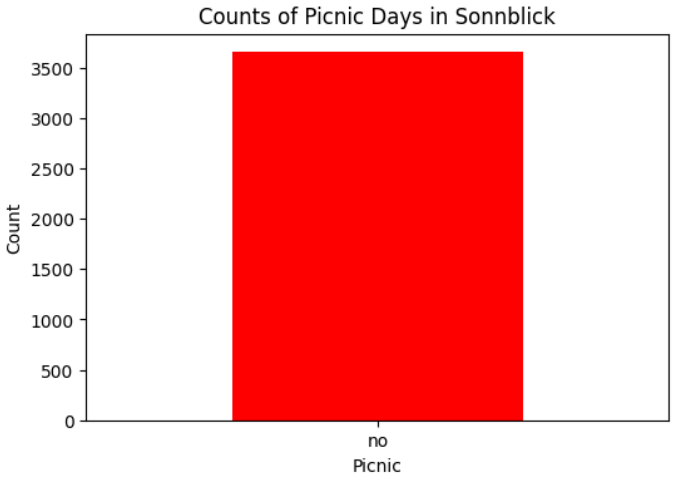
`_picnic_label` is a binary feature and represent the outcome we want to estimate, whether a picnic is possible or not.

Since the target variable represents distinct categories, the problem is formulated as a **classification task**. Therefore, classification algorithms will be employed to predict the category labels based on the input features.

To get a better feel for the dataset, we provide some histograms of the target label.



For Oslo there is a dominant frequency of "no picnic", as almost every day is classified as not suitable for picnics (about 83%). Conversely, for Perpignan there is a slightly dominant frequency of "yes picnic", with a balance of 52%.



Observing the chart, one can infer that no picnic can be held at the Sonnblick because it is an extreme high-mountain area, over 3,000 meters, with very low temperatures even in summer and snow present almost year-round. The area hosts an important meteorological observatory precisely because the extreme weather conditions are ideal for climate study.

Contrary to the data from Perpignan, the Sonnblick data are highly imbalanced: all of its observations are "no-picnic." However, because Sonnblick makes up only a small fraction of the overall "no-picnic" cases in the dataset, a global model still learns positive ("yes-picnic") patterns from other cities and can therefore temper its tendency to over-predict "no" for Sonnblick.

Some effective strategies for managing **class imbalance** include evaluating models with F1-score, precision and recall rather than accuracy alone.

Chapter 3

Data Cleaning

3.1 Some issues

While compiling some models, we noticed certain errors and/or omissions. First of all, we immediately observed that the column for the city of Rome was missing in the labels. Consequently, we decided to exclude its features from the training of our ML models, as we adopted supervised learning techniques that require labels for training.

Meanwhile, for the city of De Bilt, we encountered a mismatch issue between the city name used for features (labelled as `DE_BILT_features`) and the name used in the labels (labelled as `DE_picnic_weather`). This caused significant problems in our models because the identifier `BILT_features` was interpreted as a new weather feature, causing models to treat it on par with `precipitation`, `cloud_cover`, ...

We resolved this issue by changing the city name in both the dataset and labels, standardizing the naming to `DEBILT_features` and `DEBILT_picnic_weather` to ensure consistent matching.

3.2 What is data cleaning?

Data cleaning is the process of detecting and correcting (or removing) corrupt or inaccurate records from a record set, table, or database and refers to identifying incomplete, incorrect, inaccurate or irrelevant parts of the data and then replacing, modifying, or deleting the dirty or coarse data.

Data cleaning consists of:

- **Handling Missing Data**: Identifying and addressing missing data in the dataset. Strategies include *imputation*, where missing values are replaced with an estimated value, or *deletion* of rows or columns with missing data.
- **Detecting and Treating Outliers (Irregular Data)**: Identifying and managing outliers that may misrepresent the analysis. Outliers can be treated through *trimming* (removing extreme values), *capping* (limiting extreme values by substitution with average values), or applying *data transformations* (e.g., logarithms, roots, etc.).
- **Handling Categorical Features**: Converting categorical variables into a format usable for analysis to transform categories into numbers.
- **Standardizing Data**: Rescaling data to a certain range or distribution to improve the performance of models.

The dataset on which we conducted our study did not contain raw data. In fact, a first data cleaning had already been performed: any variable (column) with more than 5% invalid entries (marked as "-9999") was completely removed.

After filtering, the dataset comprises **165 variables** (instead of 198 that we expected) observed over 3654 days for each of the 18 locations.

3.3 Handling Missing Data

An essential step in the data-analysis workflow is identifying any missing values. Such gaps can occur for a variety of reasons - faulty sensors, incomplete recordings or simple data-entry mistakes. By pinpointing which features contain nulls and examining their frequency and pattern, we can gauge how much they might deviate our model's results.

The dataset provided had already been processed for missing data: variables with $\leq 5\%$ invalid entries have been retained, but in these cases each invalid or missing value has been replaced by the mean of that variable computed over the valid entries.

Note: Replacing missing values by the mean alters the original data; this step was applied because the resulting dataset is intended primarily for training purposes.

However, we wanted to check that there was no missing data for each recording using the following Python code, that produces a list showing the percentage of missing values for each of the features.

```
import numpy as np

missing_columns = []
for col in df.columns:
    if col in ['DATE', 'MONTH']:
        continue
    pct_missing = np.mean(df[col].isnull())
    print('{} - {}%'.format(col, round(pct_missing*100)))
    if pct_missing > 0:
      missing_columns.append(col)

if not missing_columns:
    print("\nNon ci sono dati mancanti nel dataset.")
```

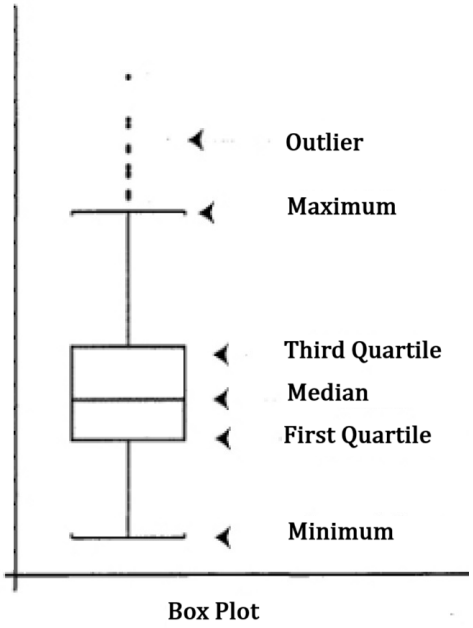
`df[col].isnull()` returns a series of booleans: `True` with a value of `NaN` (*missing data*), `False` otherwise.

`np.mean(df['A'].isnull())` gives $\frac{\text{number of True}}{\text{total elements}} = \frac{\text{number of missing data}}{\text{total elements}}$. In our case, it turns out that number of True=0, so there aren't missing data, as we wanted.

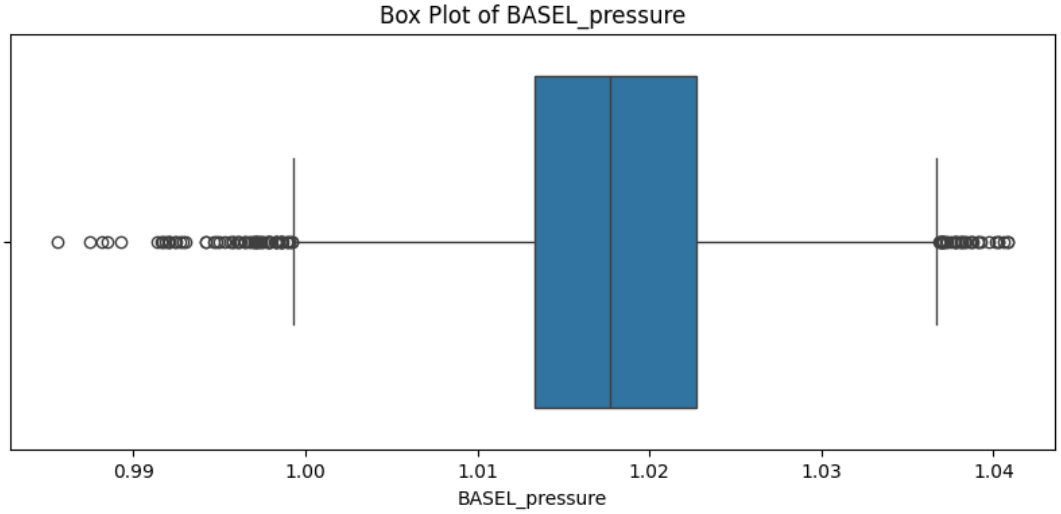
3.4 Treating Outliers

Outliers are data that is distinctively distant from other observations. They could be real outliers or mistakes. Machine learning algorithms are sensitive to the range and distribution of attribute values. Data outliers can spoil and mislead the training process resulting in longer training times, less accurate models and ultimately poorer results.

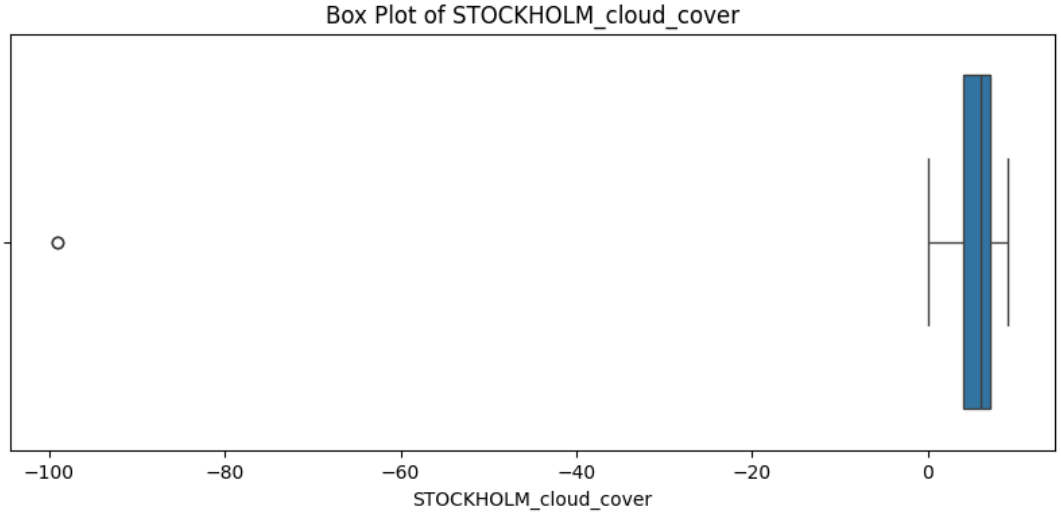
To analyse the outliers, we used box plots. A box plot is a graphic representation of data based on a five-number summary: minimum, first quartile, median, third quartile and maximum. It is represented by a rectangle, oriented horizontally or vertically, divided into two parts, from which two lines extend. The rectangle, i.e. the box, is bounded by the first and third quartiles, and divided inside by the median. The lines are bounded by the minimum and maximum values. The circles indicate the outliers.

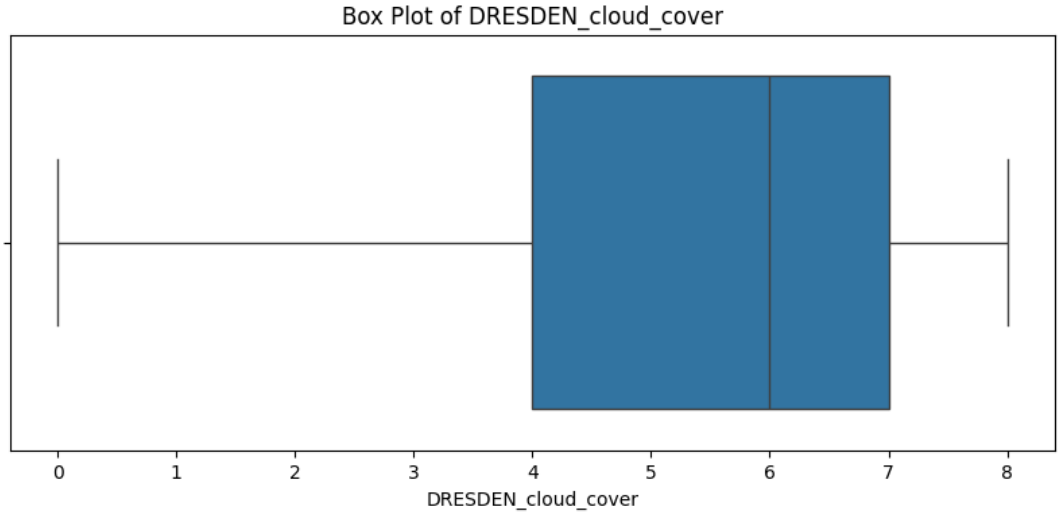


Analysing the features, we searched for potential outliers; we observed that there are some minor outliers since they deviate only slightly from the mean values in the blue rectangle.



Additionally, an extremely significant case is certainly the cloud coverage of Stockholm, which has two values equal to "-99" - a clear error since the feature is categorical and allows values between 0 and 8. Below, we will illustrate the chart and compare it to an error-free case.





We handled the compromised values of `STOCKHOLM_cloud_cover` according to solutions used also for missing data. In our case, since there were only 2 compromised values out of 3654, we replaced the outliers with the “mode“ (the most frequent value) and did not delete the entire column. In our dataset, the most frequent value in the `STOCKHOLM_cloud_cover` column is 8, with a frequency of 856 out of 3654. (We refer to it as the “mode” because the `cloud_cover` feature is categorical.)

Then, we checked that the data fell within an admissible range: for example, humidity must be between 0 and 100% (inclusive), precipitation cannot take negative values, and similar constraints were applied to other weather features.

We proceeded to correct the outliers of the features `STOCKHOLM_pressure` and `TOURS_pressure` because they were less than or equal to 0 (and the pressure can't be negative), and we replaced them with the mean value of the data from the other observations. (We use the mean and not the mode as before, because the `pressure` feature is numeric.)

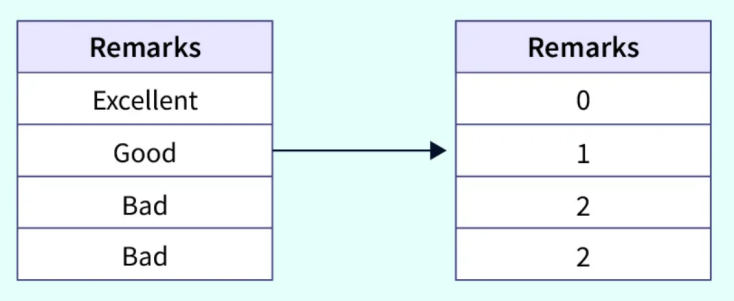
3.5 Handling Categorical Features

From the data schema we identified three categorical columns:

- `_MONTH`: month names ("January", "February", ..., "December")
- `_cloud_cover`: textual sky-cover categories (e.g. "not covered", "partly covered", ..., "completely overcast")
- `_picnic_label`: binary class label ("True"/"False")

These features must be transformed into numerical values before they can be used in machine learning models. Two of them were already provided to us in encoded form, and we encoded the other one ourselves.

Ordinal encoding is a data preprocessing technique used to convert categorical data into numerical data while preserving the inherent order or ranking of the categories. This is done by assigning a unique integer value to each category, where the numerical values reflect the ordinal relationship between the categories.



Similarly, **label encoding** converts each category into a unique integer. Technically it performs the same operation as ordinal encoding (mapping categories to integers); the difference is that ordinal encoding implies a meaningful order among the categories, whereas label encoding does not.

`_MONTH` We retained the original numeric coding (1–12), i.e. an ordinal encoding that reflects the natural order of months. This approach is especially well-suited for tree-based models.

`_cloud_cover` Here as well we kept the original coding using ordinal encoding, assigning each category an integer from 0 to 8 (0 = "not covered", ..., 8 = "completely overcast"), since the classes follow a logical progression of coverage.

`_picnic_label` This is our target variable. Since it was provided as True/False, we applied label encoding to transform them into 1/0 (1 = picnic possible, 0 = picnic not possible).

After encoding, all variables are in numeric form and ready to be used in our classification models.

3.6 Standardizing Data

The variables in the dataset are *not* all completely standardized, to keep the reported units and magnitudes as intuitively interpretable as possible. Nonetheless, these transformations help reduce extreme disparities in the raw scales, making the data more suitable for many machine or deep learning methods without requiring additional preprocessing. In models where standardization is necessary, we will apply it using the StandardScaler function.

StandardScaler is used to standardize the input data. It belongs to the `sklearn.preprocessing` module of `scikit-learn`, an open-source machine learning library for the Python programming language. Its purpose is to transform each variable so that it has a mean of 0 and a standard deviation of 1. In practice, for each column (feature) x in our dataset, it applies the formula:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

where:

- μ is the mean of the values of x computed on the training set,
- σ is the standard deviation of x computed on the training set.

Standardization transforms the data such that the mean of each feature becomes zero and the standard deviation becomes one. This ensures that the data points have a balanced scale, which is critical for many methods, especially those sensitive to differences in feature scales, such as:

- **Principal Component Analysis (PCA)**, a linear dimensionality reduction technique that computes the covariance matrix of features: without scaling, features with larger variances would drive the principal components entirely.
- **t-Distributed Stochastic Neighbor Embedding (t-SNE)**, another dimensionality reduction method that maps the data into a lower-dimensional space while keeping similar points close. Unbalanced feature scales would distort these similarity estimates.
- **k-Nearest Neighbors (K-NN)**, where neighbor selection is based on Euclidean (or other) distances, and unscaled features with larger ranges would otherwise dominate the distance computation.
- **Neural Networks**, where many activation functions (sigmoid, tanh) become "flat" when the input is too large or too small, losing the ability to distinguish changes. With zero-centered data and limited dispersion, activation functions operate in their maximum sensitivity range.

Chapter 4

Feature selection

It is not sufficient to simply "throw" all available variables into the algorithm and hope it figures out what is truly important. In fact, doing so almost always leads to sub-optimal results for several reasons:

- Each additional variable increases complexity: the model becomes heavier to interpret and explain. With too many features, it becomes practically impossible to understand which factors are actually driving the prediction.
- Datasets often contain columns that bring no useful information for weather forecasting (for example, the `_DATE` column in our case). Including them introduces noise, distracting the algorithm and reducing prediction quality.
- A model with thousands of features requires much more time for training and for producing predictions in production. In real-time scenarios or on resource-constrained devices, this slowness can become a bottleneck.
- Feeding the algorithm variables full of errors or filled with missing values will inevitably produce poor-quality forecasts. It is better to invest time in cleaning and selecting the most informative features.

To make a good selection of features, we use the *Pearson's correlation*, that is a **filter based method**. A filter based method evaluates each input variable independently of any particular learning algorithm. In other words, it "filters" features before we even train our model, using simple statistical criteria to decide which features are likely to be informative.

4.1 Pearson's Correlation

We check the absolute value of the Pearson's correlation between the target and numerical features in our dataset. We keep the top features based on this criterion. The metric used in Pearson correlation is the *Pearson correlation coefficient*, defined by the following formula:

$$r = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2 \sum_{i=1}^{n}(y_i - \overline{y})^2}} \in [-1, 1]$$

where

- x_i and y_i are the individual data points for the variables x and y,
- $\bar{x}$ and $\bar{y}$ are the means of the variables x and y,
- n is the number of data points.

Since the important thing is to identify the strongest relationships (positive or negative), we take $|r|$ and sort (with the command `sorted_avg_correlations`) the features in decreasing order of absolute correlation. We then select the top n features in this ranking to train the model. However, it's important to note that the Pearson's correlation only captures linear relationships.

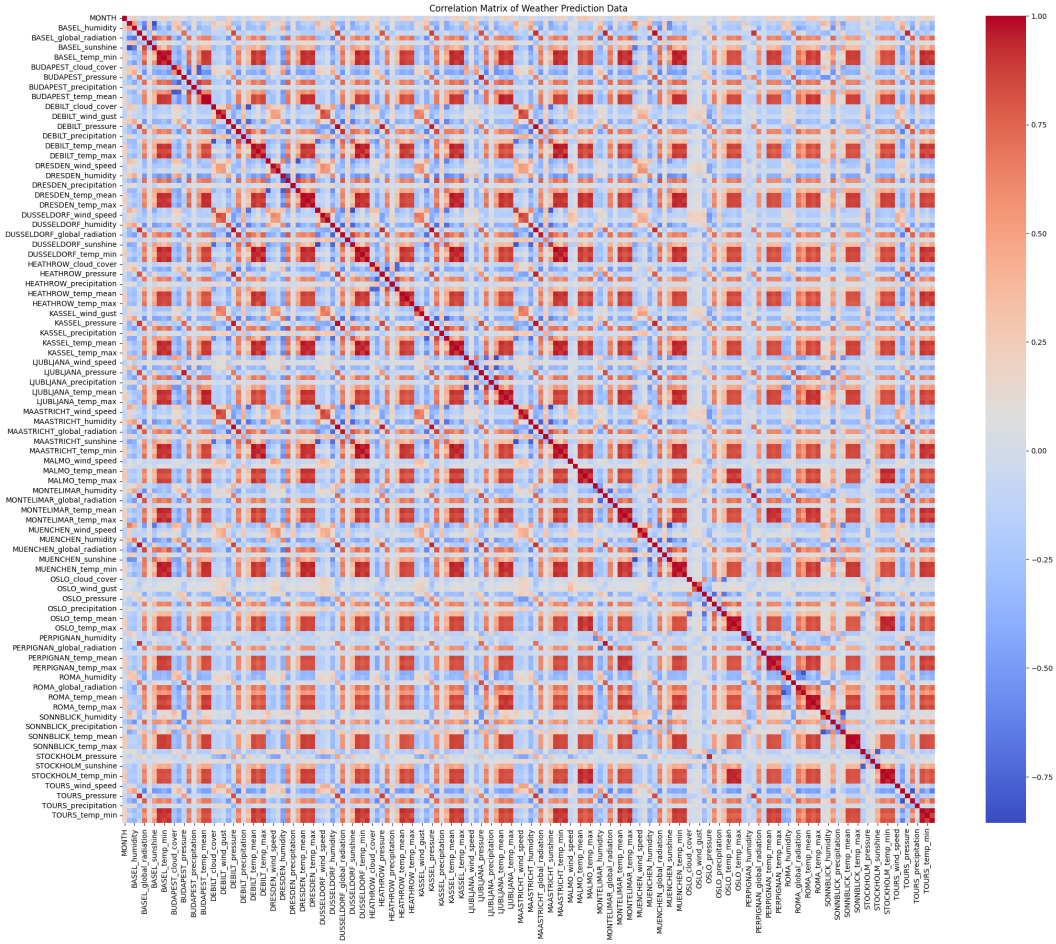
The *correlation matrix* is a powerful statistical tool for exploring relationships among the dataset's features. The correlation matrix is a square table in which both rows and columns represent the same features. Each cell (i, j) the Pearson correlation coefficient between a pair of variables i and j. The matrix has the same features listed in identical order along both axes, so on the main diagonal, we always find the value 1, because each variable is perfectly correlated with itself.

- 1 indicates a perfect positive linear relationship
- -1 indicates a perfect negative linear relationship
- 0 indicates no linear correlation

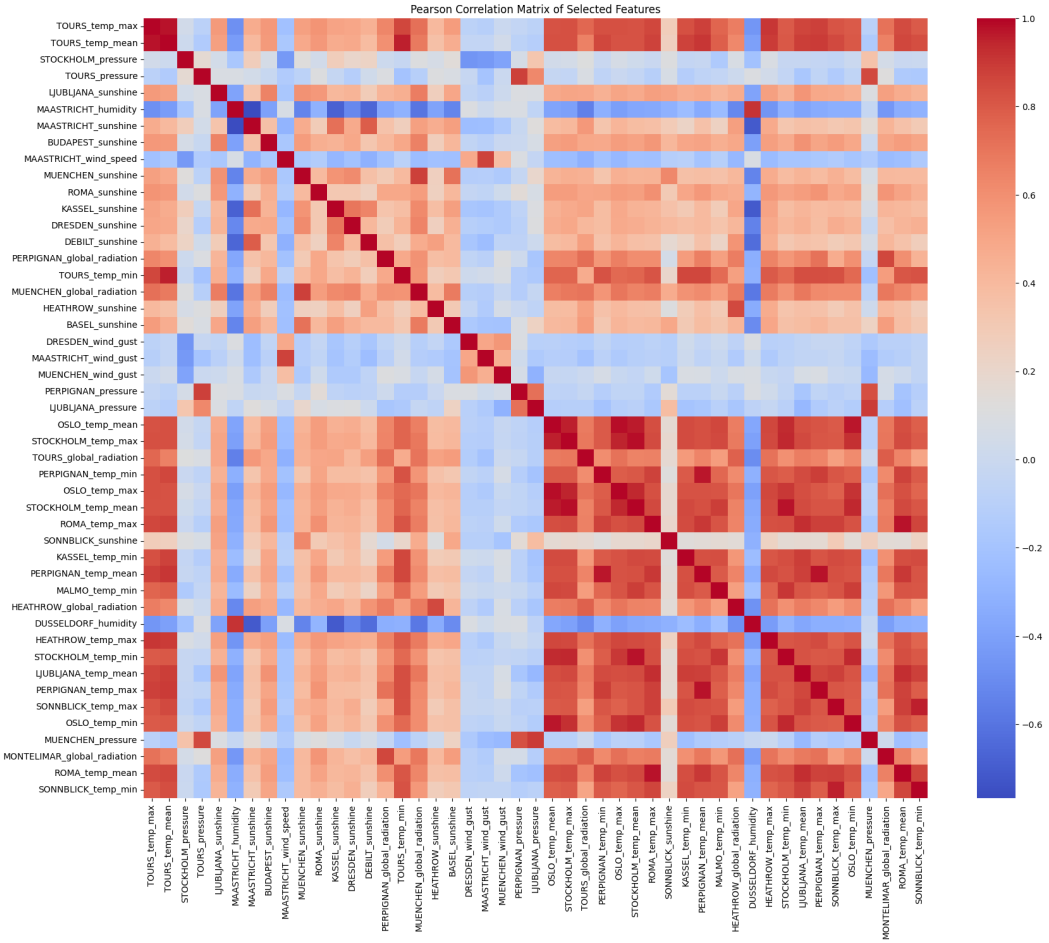
The correlation matrix is fundamentally important because, by comparing all features with one another, we can remove highly correlated pairs, avoiding redundancy and instability in linear models. We quickly obtain an immediate

overview of the existing relationships, enabling informed decisions about which features to include, transform, or discard.

Below we show the correlation matrix of the whole dataset.



We computed a Pearson correlation matrix to assess the linear relationships between features. However, due to the large number of variables (165), the matrix appeared difficult to interpret, with many redundant correlations.



To improve clarity and reduce redundancy, we applied a correlation threshold of 0.8: if the absolute value of the correlation between two features exceeded this threshold, one of the two was considered redundant and removed. This can be done with the `highly_correlated_pairs` structure, that is a DataFrame indexed by feature pairs (i, j) whose mutual Pearson coefficient exceeds a chosen threshold. It reveals redundant variables, which are inserted into an `excluded_features` set.

As a result, we reduced the number of features from 165 to 48, eliminating 117 highly correlated variables.

Chapter 5

Dimensionality reduction

We would like to find an optimal feature-subset, where the features are independent, have enough variance and are not too many. Dimensionality reduction is a technique used to reduce the number of variables in a dataset while retaining as much information as possible. We use two main methods:

- PCA (*Principal Component Analysis*)
- t-SNE (*t-distributed Stochastic Neighbor Embedding*)

5.1 PCA

Our aim is to create an ideal set of features with these properties:

High Variance: features with greater variance tend to carry more useful signal.

Uncorrelated: highly correlated features offer redundant information and can even degrade performance.

Compact Number: we need far fewer features than observations. Too many features relative to the number of data points leads to overfitting and poor generalization on new data.

Principal Components Analysis (PCA) is a dimensionality reduction technique and naturally delivers this ideal feature set. It transforms the original variables into a smaller number of principal components that are

- **ranked by decreasing variance** (so the first captures the most signal, the second the next most, and so on),
- **mutually uncorrelated**,
- easily truncated to retain only the **most important components**, keeping the feature count low while preserving most of the information.

PCA tries to find another representation of the data (another set of features called Principal Components), such that the features in this representation have the highest possible variance and lowest possible covariance.

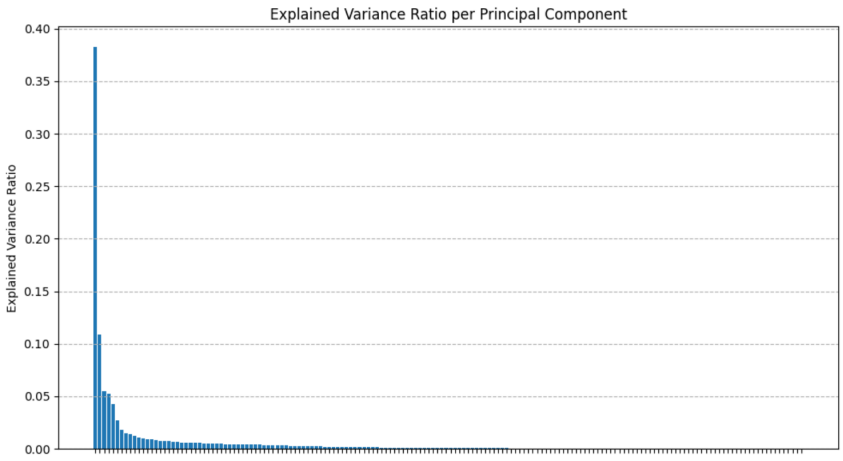
We obviously want to be able to explain as much variance as possible but at the same time we want to reduce the number of dimensions.

How does PCA do this? After computing the covariance matrix, which shows how the features vary together, it determines its eigenvalues and eigenvectors.

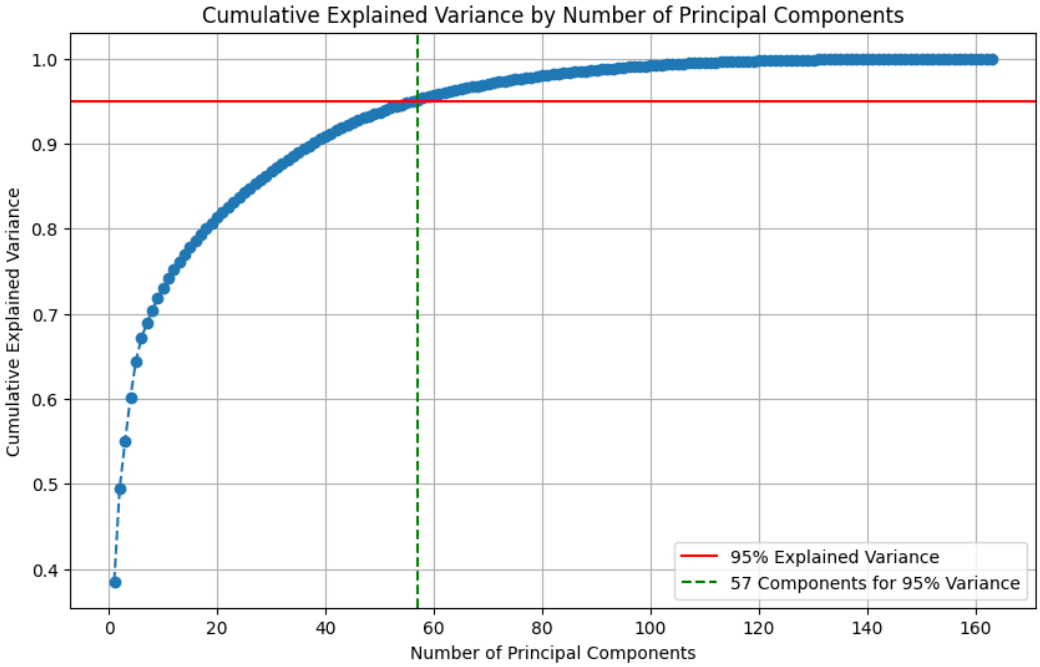
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

eigenvalues, eigenvectors = np.linalg.eig(covariance_matrix)
diagonalized_matrix = np.diag(eigenvalues)
print("Diagonalized Covariance Matrix (Eigenvalues on the diagonal):")
display(diagonalized_matrix)
sorted_eigenvalues = np.sort(eigenvalues)[::-1]
```

Eigenvalues represent the amount of variance explained by each principal component, while eigenvectors define the new directions (known as principal components) along which this variance is maximized. The diagonalized matrix is then constructed, and the eigenvalues are sorted in descending order using `np.sort(eigenvalues)[::-1]` to ensure that the component explaining the greatest amount of variance appears first.

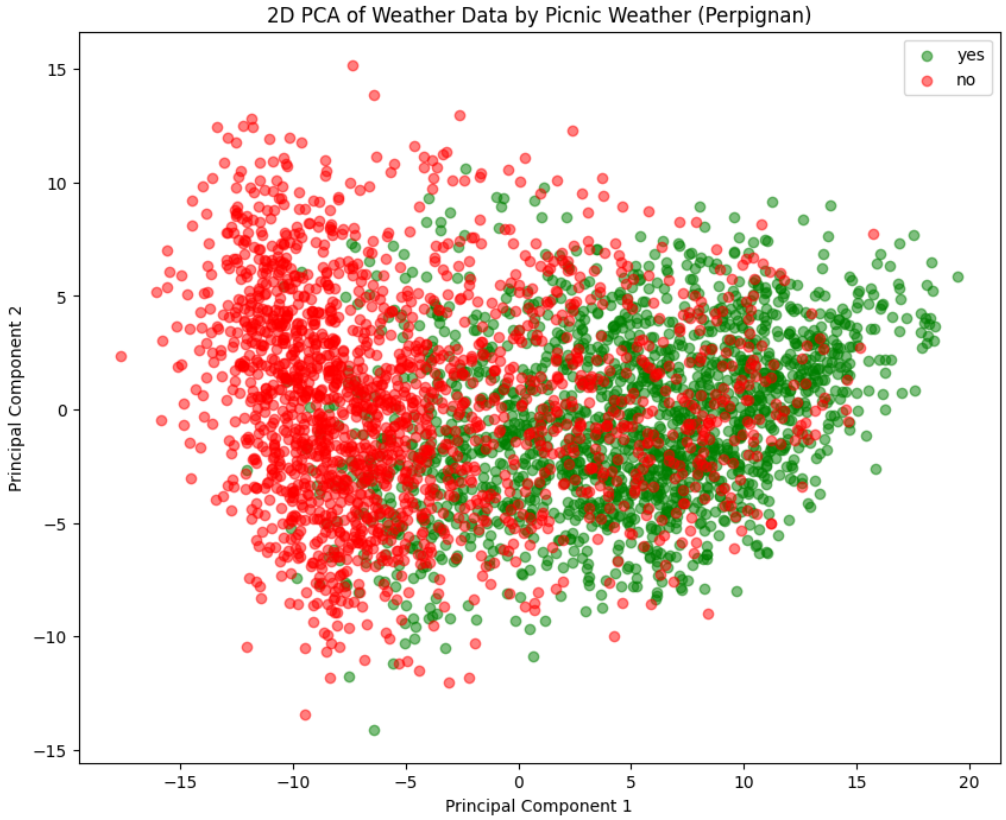


So, which components explain the most variance?



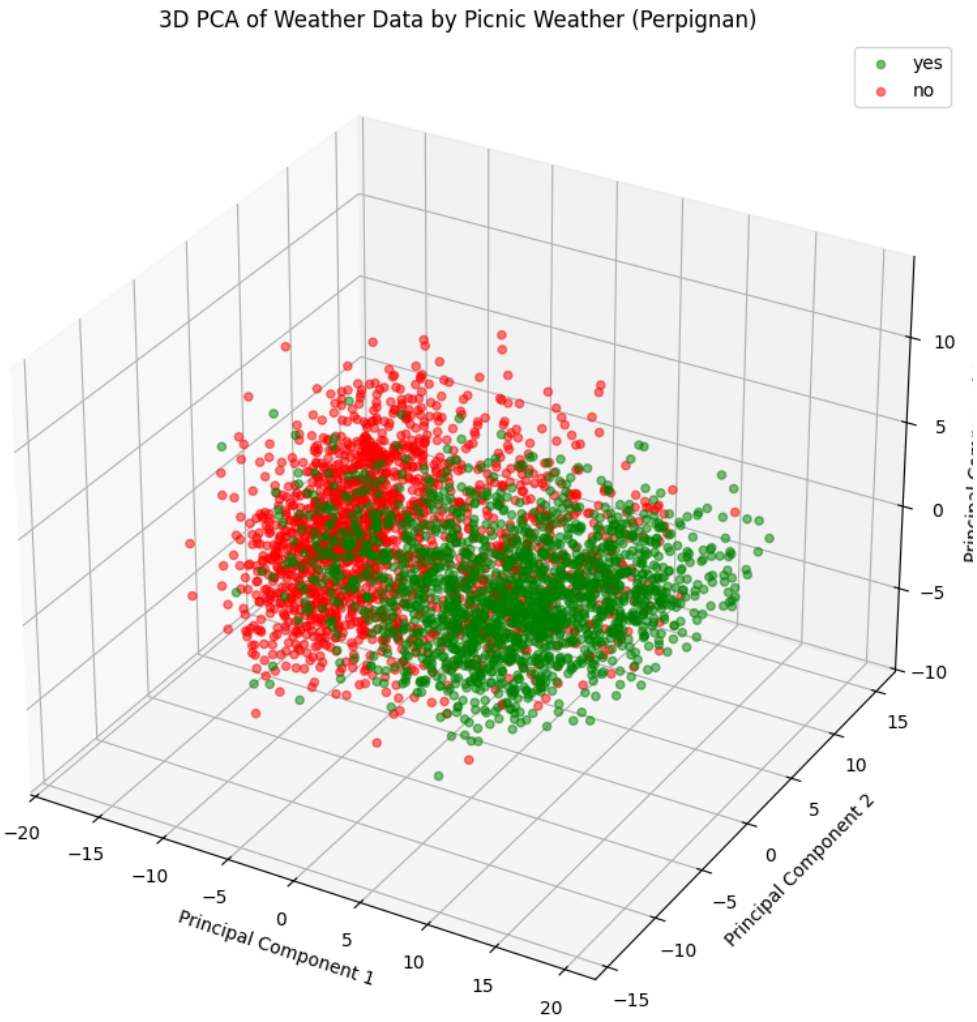
From the chart, we can see that 57 principal components are needed to explain 95% of the variance in the dataset. This means we can reduce the dimensionality of the original dataset from 164 features to 57 principal components, while still retaining most of the information.

We generated PCA plots for each city in our dataset. Here's the one for Perpignan.



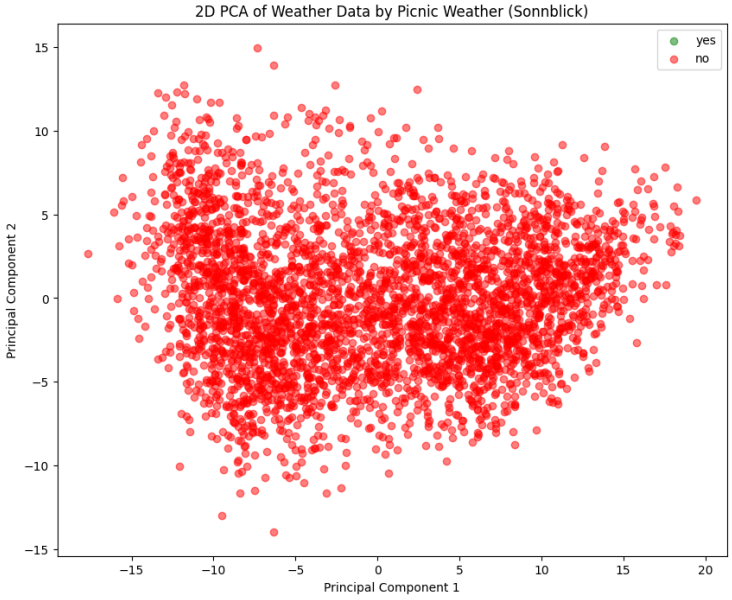
In the 2D plot that we get from the first two principal components (PC1 and PC2), we can see points labeled "picnic yes" (green dots) mostly grouped on one side, and points labeled "picnic no" (red dots) on the other side, forming two separate *clusters* (groups of elements that share similar characteristics or behaviors).

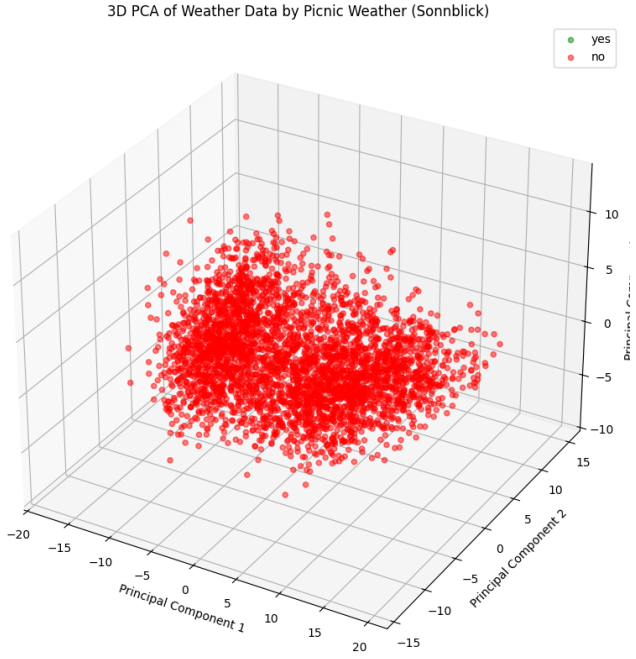
We can see that PC1 (the horizontal axis) separates the two groups quite well: the "yes" days have, on average, higher PC1 values. It is noted that the data is well divided by similarity and it means that this transformation has captured the most important information that differentiates those who have a picnic from those who don't. So, the first two principal components do a good job summarizing the differences between the two classes using only two variables instead of many.



Furthermore, we show a 3D plot (obtained using the first 3 PC) to look better the distinction. Looking at the charts, the data would seem to be mostly divided by a separator plane, however some different elements can be noted on the sides.

We also report the case of Sonnblick due to its peculiarity.

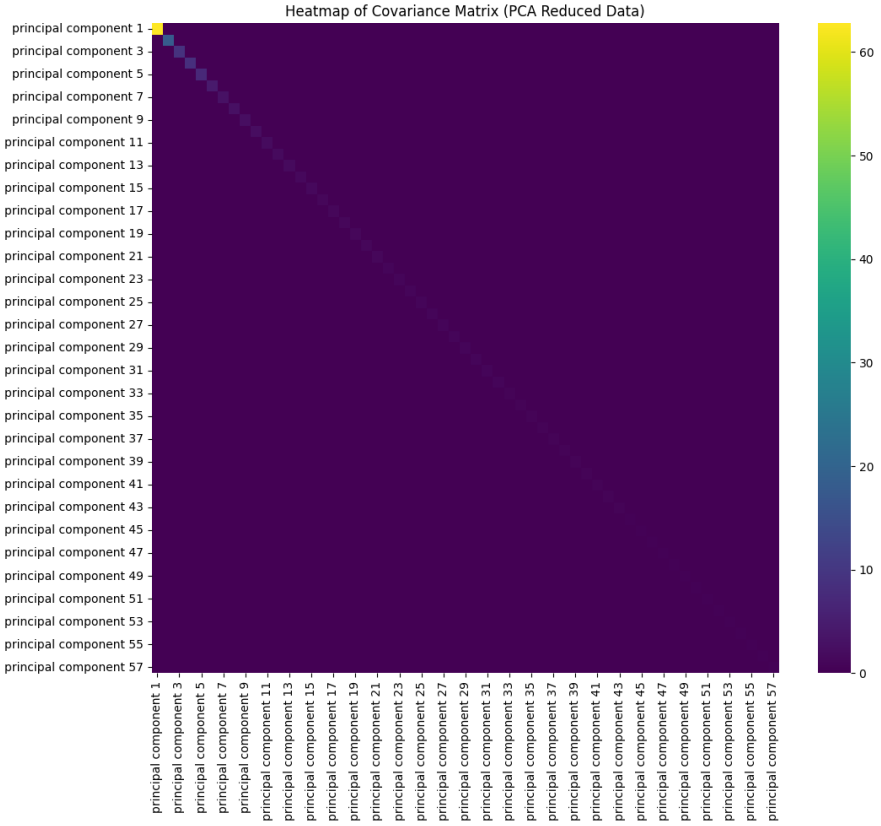




Since all our data points for Sonnblick city belong to the same class ("no picnic"), PCA doesn't show any class-based separation, but the result is one dense cluster.

The **covariance matrix** is a square matrix that shows the covariance between each pair of variables - in this case, the principal components. Covariance is a measure of how two variables change together: positive covariance indicates that as one variable increases, the other tends to increase as well; a negative covariance means that as one variable increases, the other tends to decrease. A covariance close to zero suggests that there is little to no linear relationship between the two variables.

To ensure that PCA has been performed correctly and to see the distribution of variance across the new axes (the principal components), we can visualize the covariance matrix.



In this heatmap of the covariance matrix (computed after applying PCA), the elements on the main diagonal represent the variance of each principal component. These diagonal values are the brightest in the heatmap and indicate how much variability (i.e. information) is captured by each principal component. The first principal components (PC1, PC2, etc.) typically have the highest variance (and thus the largest values on the diagonal). The off-diagonal elements show the covariance between different pairs of principal components.

A key property of PCA is that the principal components (eigenvectors) are orthogonal (i.e., linearly uncorrelated). Therefore, in a correctly computed covariance matrix of principal components, the off-diagonal values should be very close to zero (theoretically exactly zero, with possible small deviations due to precision errors).

So, this heatmap confirms that: PCA successfully transformed the data into a new set of features (the principal components) that are linearly uncorrelated with one another. This is evidenced by the near-zero values in the off-diagonal elements of the covariance matrix. The main diagonal clearly shows how the total variance of the original dataset is distributed across the principal components, with the first few components capturing the majority of the variance.

5.2 t-SNE

Distributed Stochastic Neighbor Embedding (t-SNE) is another dimensionality reduction technique and is particularly suited for visualizing high-dimensional datasets in 2 or 3 dimensions.

Essentially, it uses the original data fed into the algorithm and attempts to best represent it using fewer dimensions.

The t-SNE algorithm models the probability distribution of neighbors around each point. Here, the term “neighbors” refers to the set of points closest to each point.

The method used is quite computationally demanding and therefore presents serious limitations to the use of this technique; for example, it is recommended that, with high-dimensional data, another dimensionality reduction technique, such as PCA, may need to be applied before using t-SNE. Using the PCA code developed previously, we created a new dataset containing only 57 dimensions. We used this dataset to run t-SNE.

To perform t-SNE we use **perplexity**, a measure of the algorithm’s ability to pay attention to both the local and global structure of the data. In more technical terms, perplexity is related to the number of nearest neighbors that t-SNE considers when calculating the affinities between points in the dataset. Van Der Maaten & Hinton suggest perplexity values in the range 5 - 50. We have chosen perplexity value equal to 5 because it doesn’t lead to excessive fragmentation in the reduced graph and it doesn’t obscure the distinctions between

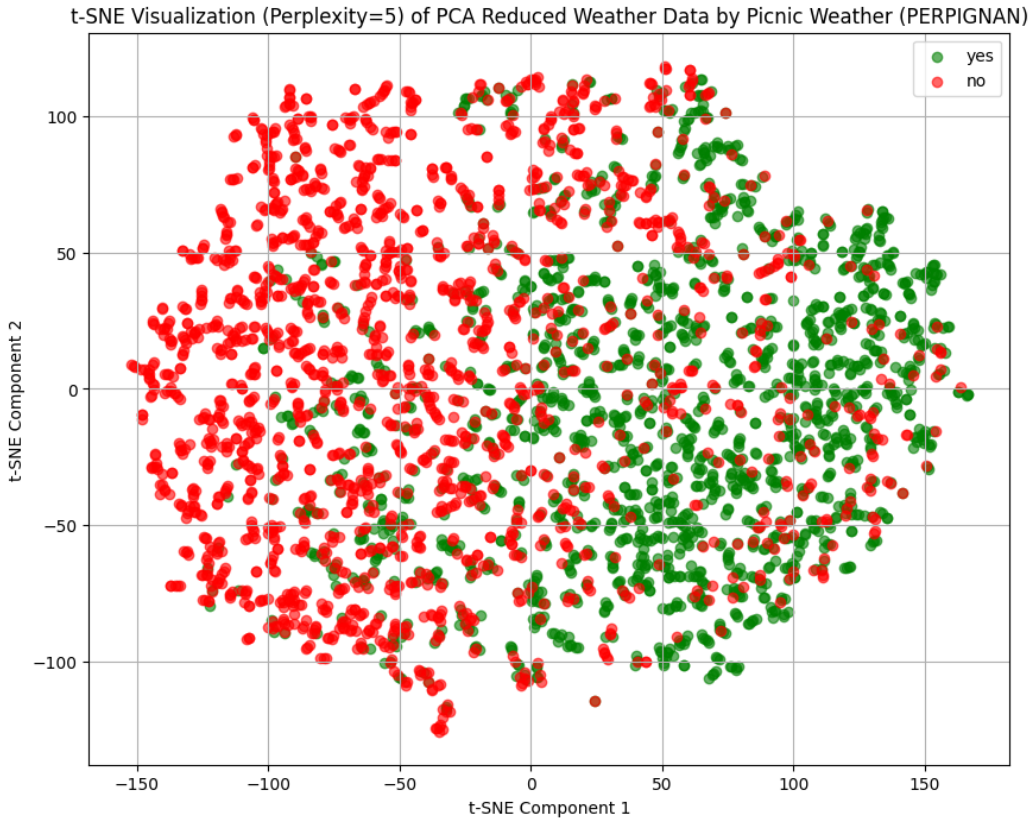
data groups.

When we compute t-SNE, we use `fit_transform` from `scikit-learn`. This single instruction carries out two essential phases:

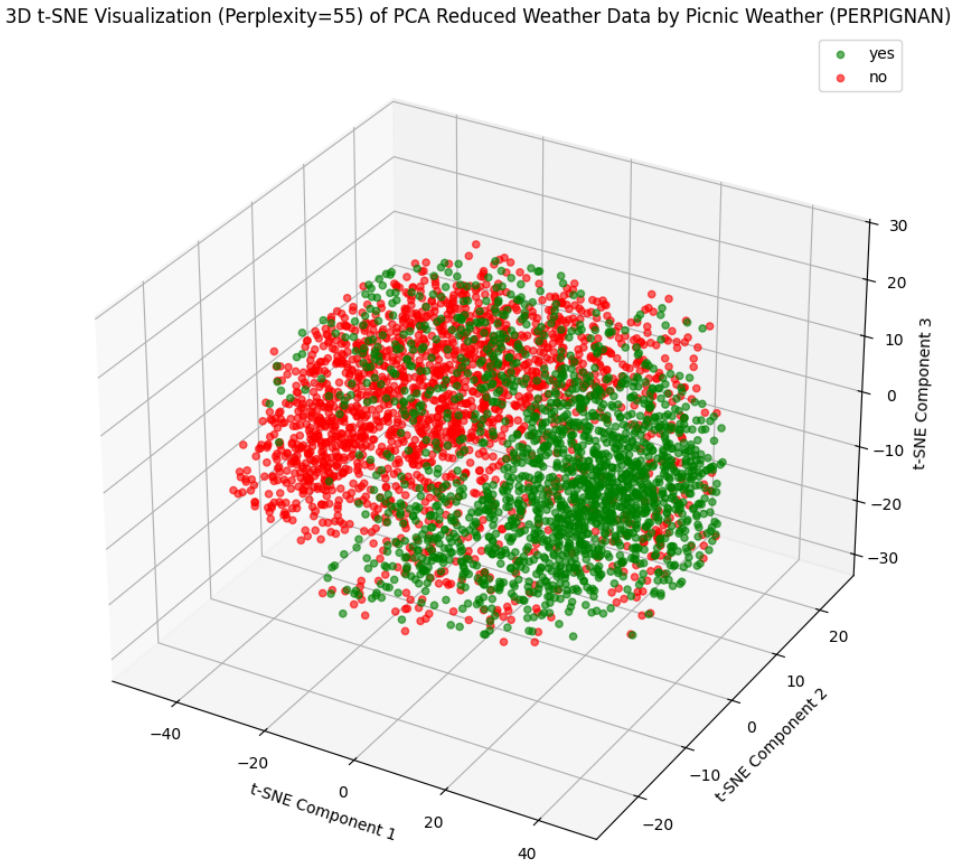
1. **fit**: t-SNE examines the high-dimensional data and computes a probability distribution over pairs of points, so that similar points have a high chance of being picked as neighbors. It then initializes a low-dimensional map and iteratively updates it, minimizing the divergence between the two distributions.

2. **transform**: once the optimization converges, t-SNE outputs the final coordinates, in which the local structure of data is best preserved. It maps the input data to the new low-dimensional representation.

We imported Matplotlib's `pyplot` module under the alias `plt`. `pyplot` is an interface that allows creating or modifying the current figure with each call. Using `plt.scatter`, we then generated a scatter plot using the t-SNE Component 1 values as x-coordinates and the t-SNE Component 2 values as y-coordinates, which were computed from the PCA-reduced data.



In this 2-D chart we can see a separation between the samples, even if the two groups are contaminated from points that belong to the other group. To visualize the points in a better way, an idea can be to plot a 3-D chart.



The 3-D chart confirm with more clarity what we have observed in the 2-D chart. In fact the data would seem to be mostly divided by a separator plane.

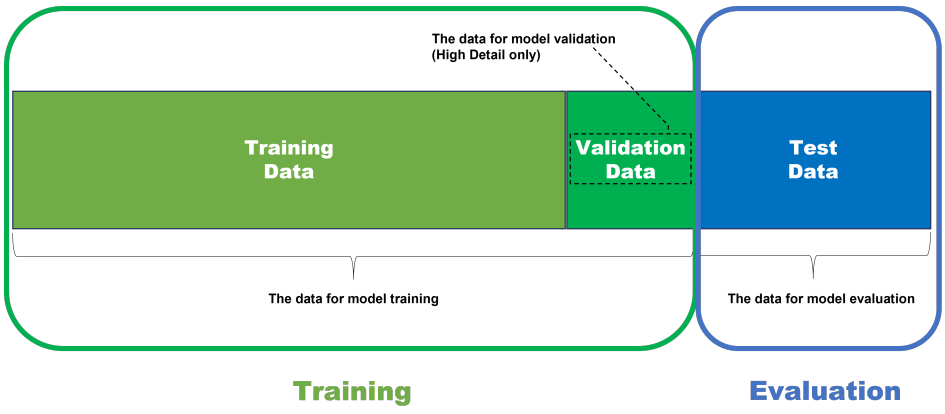
Chapter 6

Classification

After the data preprocessing, we can start the creation of the models to predict our labels, we will focus on the creation of models to solve the classification problem.

Classification in machine learning is a supervised learning technique used to categorize data into predefined classes or categories. Essentially, it's a process where a machine learning model learns from labelled data to predict the correct class for new, unseen data.

In fact, the whole dataset is usually divided into three parts: one used by the learning algorithm to learn a model (called **training data**), the second one (called **validation data**) is used to evaluate a given model and it helps to choose the best model, and the last one is used to evaluate the performance of the learned model (called **test data**).



We will use the following division ratio:

- 70% for the training set,

- 15% for the validation set,
- 15% for the test set.

We used the command `shuffled = np.random.permutation(cities)`: it takes the array of cities and returns a copy in which the elements are randomly reordered. The original array remains unchanged; the permutation is stored in a new variable `shuffled`. Then, with the following three commands, we selected the first 11 cities for training, the next 3 for validation, and the last 3 for testing.

```
shuffled = np.random.permutation(cities)

train_cities = shuffled[:11]
val_cities   = shuffled[11:14]
test_cities  = shuffled[14:17]
```

In particular, the following have been selected.

- **Train cities**: Basel, Budapest, Heathrow, Shockholm, Muenchen, Sonnblick, Maastricht, Perpignan, De Bilt, Malmo, Tours;
- **Validation cities**: Dusseldorf, Ljubljana, Montelimar,
- **Test cities**: Oslo, Dresden, Kassel.

In our case, according to the labels pic-nic yes or not, the classification is binary.

6.1 Decision Tree

A **Decision Tree** is constructed by asking a serious of questions with respect to a feature of the dataset we have. Each time an answer is received, a follow-up question is asked until a conclusion about the class label of the record. The **Gini Impurity** of a node is the probability that a randomly chosen sample in a node would be incorrectly classified if it was labelled by the distribution of samples in the node. Gini Impurity measures how impure (or mixed) the node is (how often a randomly sample would be misclassified): a Gini impurity value close to 0 means that the node is "pure" (almost all samples belong to a single class); higher values indicate a mixture of classes.

$$\text{Gini}(K) = \sum_{i \in N} P_{i,K}(1 - P_{i,K}) = 1 - \sum_{i \in N} P_{i,K}^2 \in [0, 0.5]$$

- N is the list of classes (In this case $N = \{\text{Yes}, \text{No}\}$)
- K is the category
- $P_{i,K}$ is the probability of category K having class i

At each node, the decision tree searches through the features for the value to split on to obtain the greatest reduction in the Gini impurity.

If we don't constrain the decision tree, it will adapt to the training data, which will lead to *overfitting*. To avoid this overfitting, we must constrain the decision trees during training by regularizing the **hyperparameters**. Specifically, we will adjust the following.

- **Maximum depth** is the largest length from the root to the leaf. Large depth can cause overfitting and small depth can cause underfitting.
- **Minimum number of samples** can be set to avoid the problem of having 99 samples in one of the splits and 1 in the other one, because this could be a waste of resources. Without setting a minimum number of samples, a very small value may cause overfitting, whereas a large value may lead to underfitting.
- **Maximum number of features** can be set to avoid the problem of checking the entire dataset on each one of the features, which can be

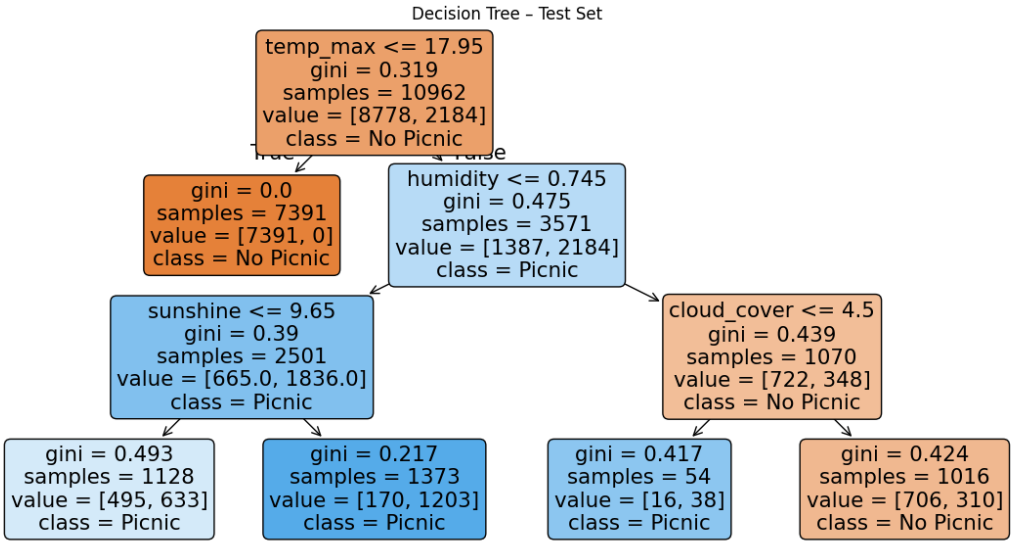
computationally expensive. If this number is high enough, we are likely to find a good feature among the ones we look for (although it may not be the perfect one). However, if it is not as large as the number of features, it will speed up the calculations signficantly.

```
# ------------------------------
#  Hyperparameters of the tree
# ------------------------------
MIN_SAMPLES_SPLIT = 10      # Minimum number of samples for split
MAX_DEPTH         = 3       # Maximum tree depth
MAX_FEATURES      = 'sqrt'  # Number of features to consider at each split
```

Therefore,

- if a node has fewer than 10 samples, no further split will be performed and it will become a leaf node;
- the tree will not grow beyond 3 levels;
- at each split, the model considers only a random subset of features of size $\lfloor\sqrt{n_\text{features}}\rfloor$.

It then repeats this splitting process in a recursive procedure until it reaches a maximum depth, or each node contains only samples from one class.



The weighted total Gini Impurity at each level of tree decreases. Let's calculate it for the first two levels.

$$I_{\text{root}} = 1 - \left(\left(\frac{8778}{10962}\right)^2 + \left(\frac{2184}{10962}\right)^2\right) = 0.319$$

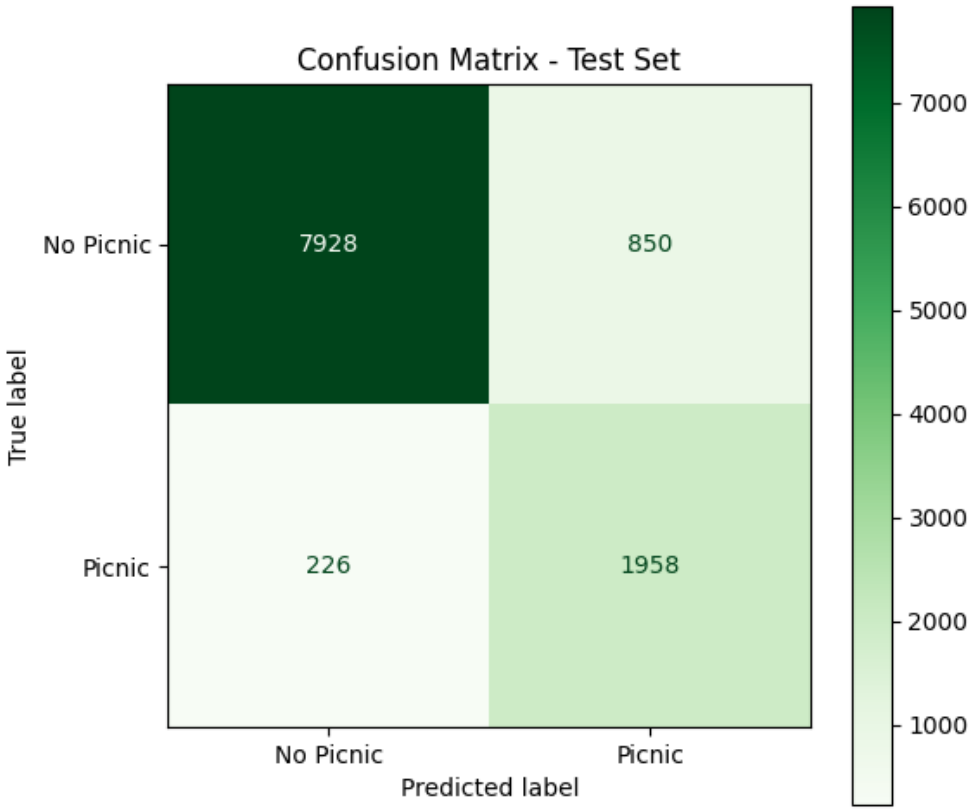
For the second layer, the Gini Impurity of each node is weighted by the fraction of points from the parent node in that node.

$$\begin{aligned} I_{\text{second layer}} &= \frac{n_{\text{left}}}{n_{\text{parent}}} \times I_{\text{left node}} + \frac{n_{\text{right}}}{n_{\text{parent}}} \times I_{\text{right node}} \\ &= \frac{7391}{10962} \times 0.0 + \frac{3571}{10962} \times 0.475 \qquad = 0.155 \end{aligned}$$

We can observe the value of weighted total Gini Impurity decreases.

The Decision Tree is one of the possible models we can use for this task. To determine the most suitable model, we calculate and compare the validation set accuracy of each candidate. The model with the highest validation accuracy will be preferred.

We plot the **confusion matrix** to observe how many examples have been correctly and incorrectly classified for each class ('No Picnic' e 'Picnic').



To be able to read this matrix, we need to know that

- the *True Positive (TP)* are the values at the bottom right;
- the *False Negative (FN)* are the values at the bottom left;
- the *True Negative (TN)* are the values in the top left corner;
- the *False Positive (FP)* are the values in the top right corner.

The overall **accuracy** is

$$\frac{TN+TP}{TN+TP+FP+FN} = \frac{7\,928+1\,958}{7\,928+1\,958+850+226} = \frac{9\,886}{10\,962} \approx 0.90$$

The model predicts correctly in 90% of the cases.

After outlining the main characteristics of the Decision Tree, we need to assess the model's reliability, that is, whether it suffers from overfitting or underfitting.
We say a model is **overfitting** when it becomes too tailored to the training data, memorizing noise and anomalies instead of learning general patterns. In essence, it is specialized solely on the training set, achieving very high performance there but significantly lower performance on the test set.
Conversely, a model is **underfitting** when it is too simple and fails to capture meaningful patterns in the data. An underfitting model shows poor performance on both the training set and the validation/test set.
A first check is to compare the **classification reports** of the training set and the test set, looking for substantial discrepancies. If we detect overfitting or underfitting, we can apply various strategies to mitigate or resolve these issues, such as tuning hyperparameters.

Our classification report includes various metrics:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1\text{-}score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

The **F1-score** is the harmonic mean of precision and recall.

Let's assume 'No Picnic' as *class 0* and 'Picnic' as *class 1*, we report the following table.

Table 6.1: Classification Report - Training Set

Class	**precision**	**recall**	**F1-score**	**support**
0	0.97	0.90	0.93	30383
1	0.75	0.92	0.82	9811
Accuracy	–	–	0.90	40194
Macro avg	0.86	0.91	0.88	40194
Weighted avg	0.92	0.90	0.91	40194

Table 6.2: Classification Report – Test Set

Class	**Precision**	**Recall**	**F1-score**	**Support**
0	0.97	0.90	0.94	8 778
1	0.70	0.90	0.78	2 184
Accuracy	–	–	0.90	10 962
Macro avg	0.83	0.90	0.86	10 962
Weighted avg	0.92	0.90	0.91	10 962

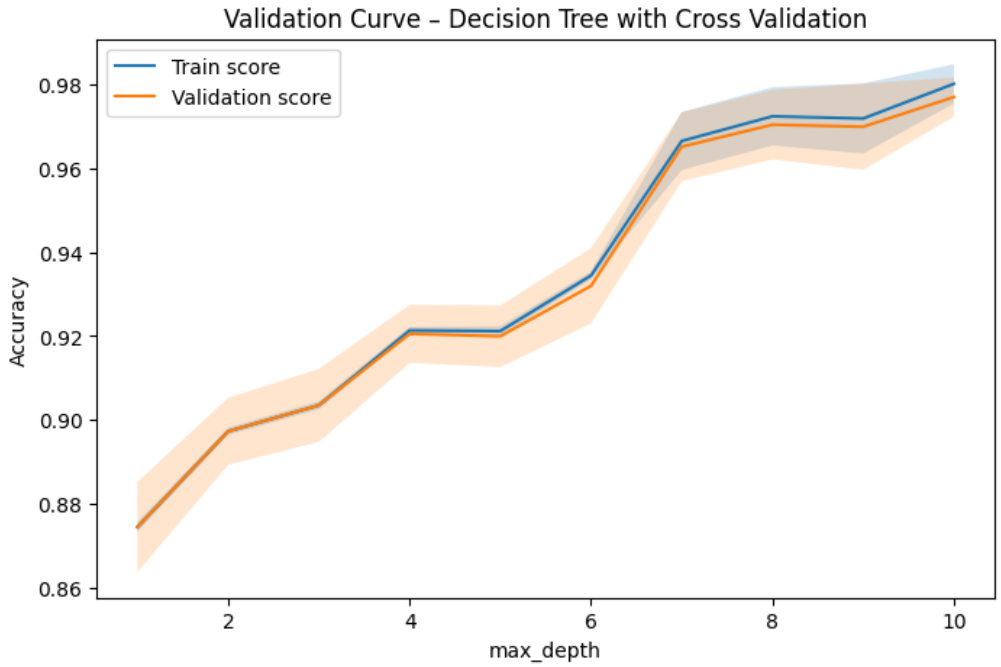
High **precision** for class 0 indicates that there are very few false positives. However, the proportion of false negatives is higher, accounting for approximately $\frac{1}{3}$ of the total samples. The **recall** correctly identified approximately 90% of the days on which a picnic is or is not possible, meaning there are few false negatives. The **F1-score** is especially useful when dealing with imbalanced classes, as in this case, where the number of "No Picnic" days (classe 0) is much higher than the number of "Picnic" days (classe 1). Weighted F1 $\approx$ 0.90-0.91 confirms a robust overall performance of the model, correctly weighing the influence of the majority class.

Generally, models tend to perform much better on the majority class because the model has seen more examples of "No Picnic" (classe 0).

We observe that there are no marked differences between the two classification reports: the values are very similar. Since there is no significant gap between the training and test sets, we can conclude that the model suffers neither from overfitting nor from underfitting and generalizes well.
The overfitting would manifest with very high scores (for example, $0.96 - 1.00$)

in the training set classification report and markedly lower scores (for example $\simeq 0.82$) in the test set report.
On the other hand, the underfitting would occur when the scores are low in both reports, for example around 0.60.

To further check for overfitting or underfitting, we can also study the decision tree's learning curve and validation curve.



The **validation curve** shows how the model's performance on the training set and the validation set changes as the `max_depth` parameter varies, helping to identify overly simple models as well as to reveal signs of overfitting or underfitting.

We distinguish three scenarios:

1. underfitting: both the `training_score` and the `validation_score` are low and close to each other.

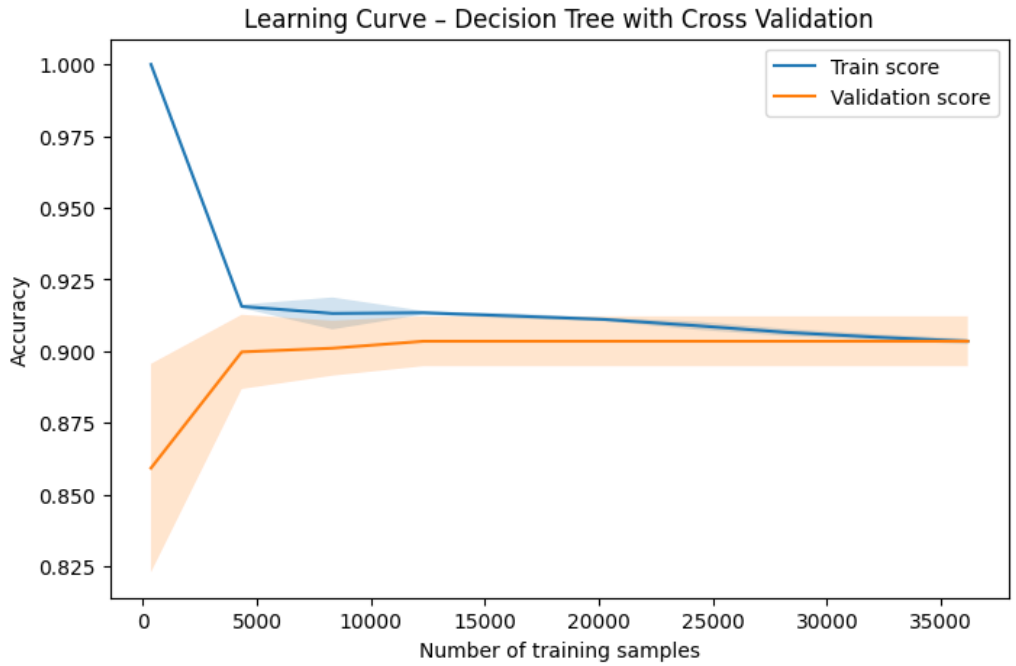
2. optimal fitting: both scores are high and close, though not perfectly equal.

3. overfitting: the `training_score` is very high (~ 1.00), while the `validation_score` is significantly lower and the gap between the two curves widens.

In our case, the validation curve confirms what we observed from the classification report: optimal fitting.

Furthermore, this plot helps us to select the optimal `max_depth`. By "optimal" we mean the value that strikes the best balance between highest accuracy and smallest gap between the training and validation curves. In our case, an optimal `max_depth` would be around 5. However, we chose `max_depth`=3 because we wanted to use the same depth for all subsequent models, ensuring an objective comparison at equal complexity. Additionally, we opted for `max_depth`=3 for the sake of simplicity and interpretability of the tree.

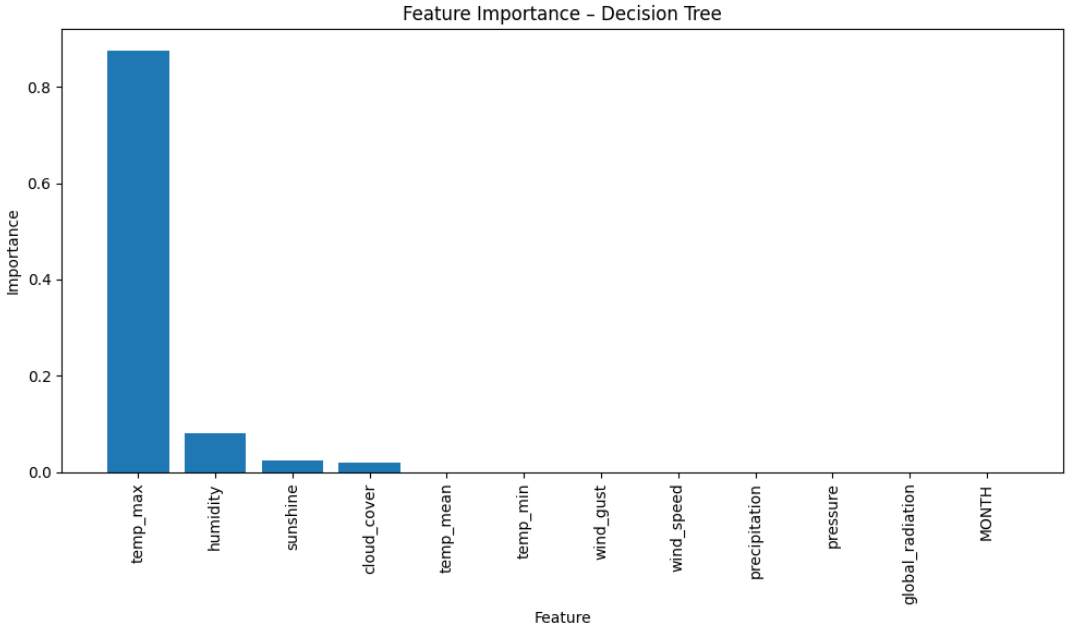
To plot the validation curve (and the learning curve we'll see shortly), we used **cross-validation**, an evaluation technique used to measure the performance of machine learning models by repeatedly dividing a limited dataset into training and validation subsets. We split our dataset into k=10 groups (or subsets) and used nine of them for training, leaving the last one as testing data. We then averaged the model across each of the subsets and finalized the model. Next, we compared the predictions to the actual labels from the test set.



A **learning curve** is a plot that shows how a model's performance (on both the training set and the validation/test set) changes as the amount of training data increases. It helps us determine whether the model suffers from underfitting or overfitting depending on the number of examples used for training.

The learning curve plot is interpreted in the same way as the one above, so here too we can identify overfitting, underfitting, or optimal fitting scenarios.

With just a few hundred samples, the training score is at 100%, while the validation score starts at around 86% with a wide uncertainty band, indicating that, at that stage, the tree is "memorizing" noise and generalizes poorly. As we add more samples, the validation accuracy quickly rises to 90% and the gap narrows (overfitting decreases because the tree sees enough data to avoid chasing occasional noise). So we find ourselves in the optimal fitting scenario.



This plot shows that few features are used and that our tree is temperature-driven.

Given the results, we can conclude that our model does not exhibit obvious signs of overfitting, despite decision trees typically suffering from it.

If the model had shown overfitting, we could have resorted to **pruning**, i.e. trimming the scattered branches that contain very few samples, thereby reducing noise and preventing the model from fitting too closely to the training data.

Alternatively, we could have switched to a more robust model, such as the *random forest*, which we will present next.

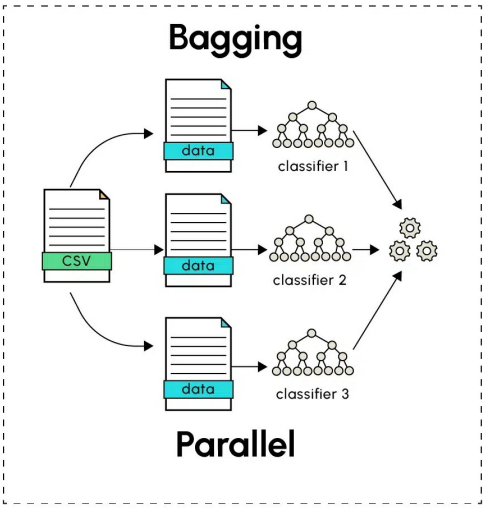
6.2 Random forest

Random forest consists of a large number of individual decision trees that operate as an ensemble, i.e. use multiple learning algorithms to obtain better predictive performance than could be obtained from a single algorithm.

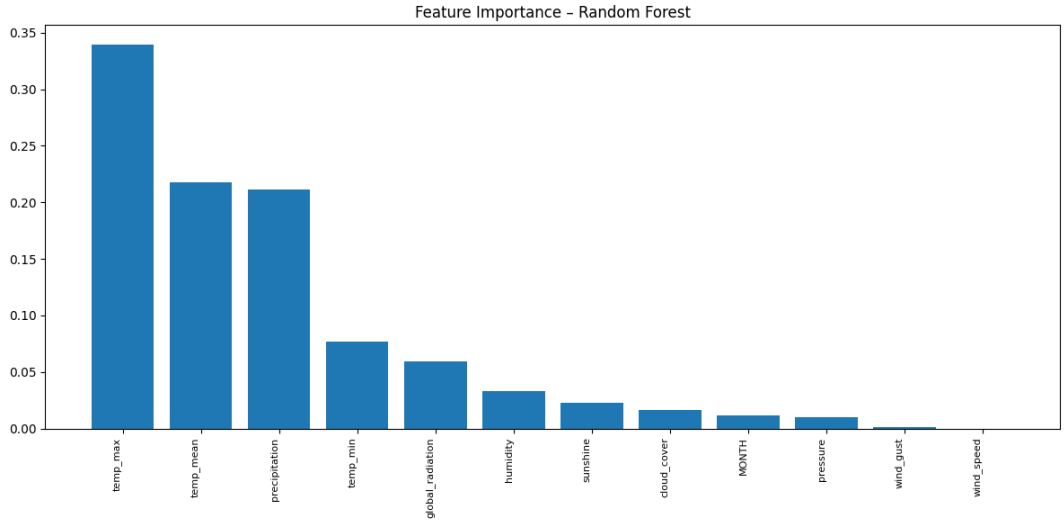
Each individual tree in the random forest spits out a class prediction and the class with the most votes becomes our model's prediction.

A large number of relatively uncorrelated models (trees) produce predictions that are more accurate than any of the single tree's individual predictions. This works because trees protect each other from their individual errors, unless they all make the same error.

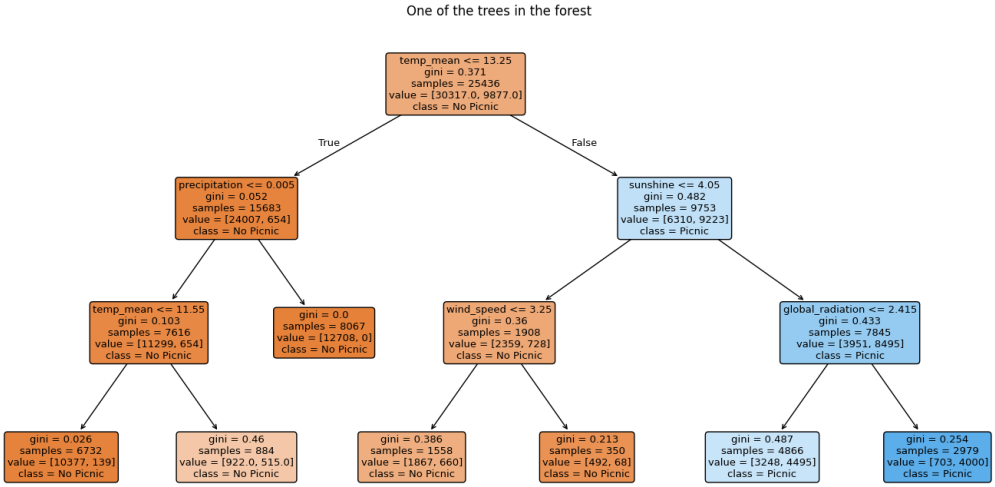
To ensure that the behavior of each individual tree isn't too highly correlated with that of the others, Random Forest uses the **Bagging method**: it works by training multiple models on different random subsets (with ripetition) of the original training dataset and then combining their predictions.



Calculating the **feature importance** with a Random Forest model, is a way of doing feature selection which tells us the features of our data that are the most helpful for the classification. For an individual decision tree, the most important features are ranked by how much they reduce the error. In a Random Forest, this procedure is applied to every tree in the forest, and the resulting values are then averaged to determine the importance of each feature.



The feature importance analysis reveals which variables the Random Forest model considered most relevant in identifying the meteorological factors that most strongly predict the possibility of having a picnic. This chart tell us that the model is driven by temperatures (min, max, mean) and precipitation.



The model is extremely accurate, as it made no classification errors. This is evident from both the confusion matrix and the classification report, where all metrics for both class 0 and class 1 are equal to 1.00, including the overall accuracy.

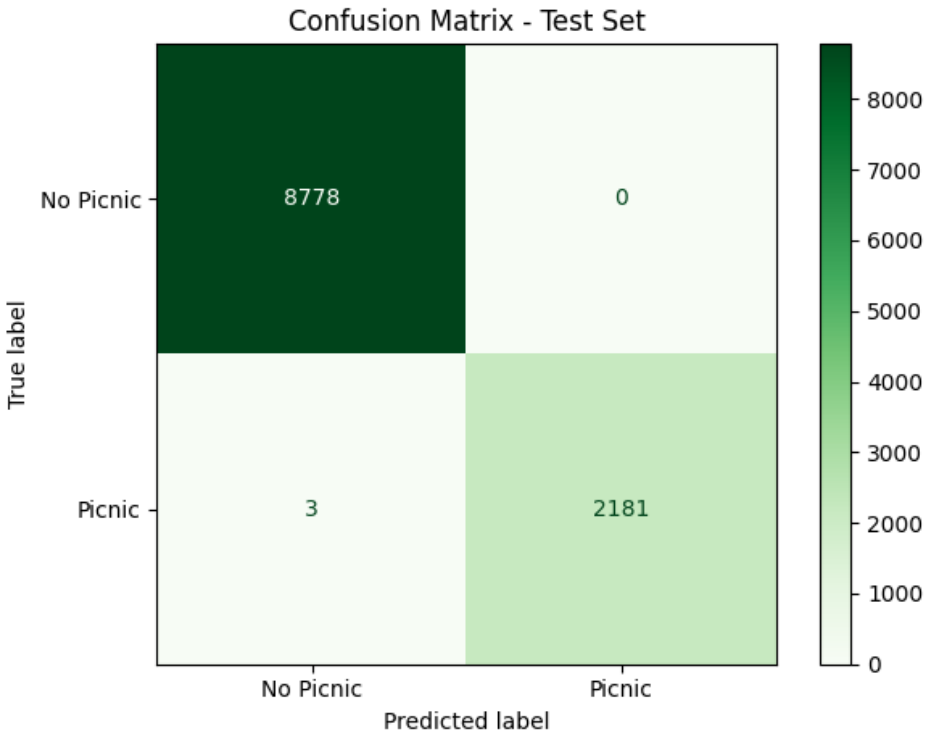


Table 6.3: Random Forest – Classification Report Training set

Classe	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	1.00	1.00	1.00	8 778
Picnic	1.00	1.00	1.00	2 184
Accuracy	–	–	1.00	10 962
Macro avg	1.00	1.00	1.00	10 962
Weighted avg	1.00	1.00	1.00	10 962

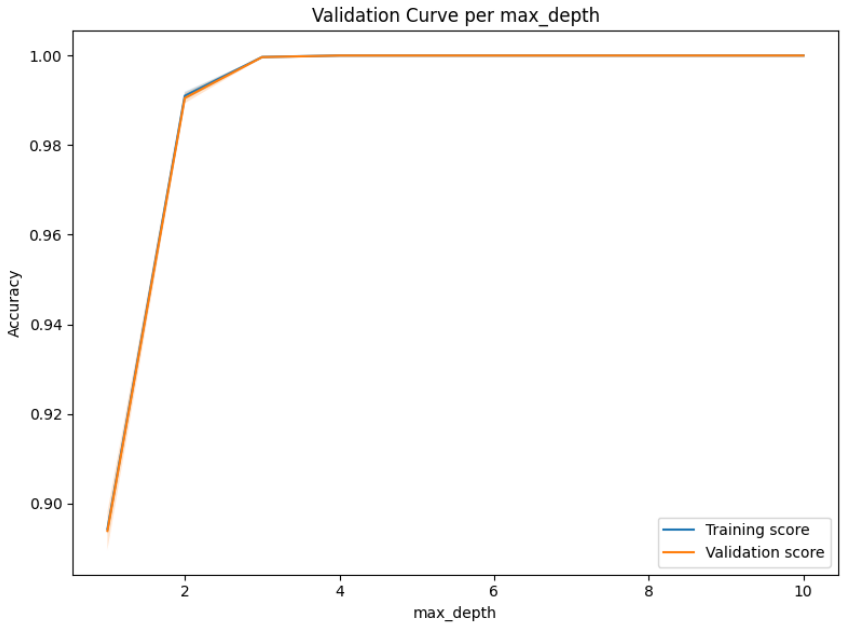
Table 6.4: Random Forest – Classification Report Test set

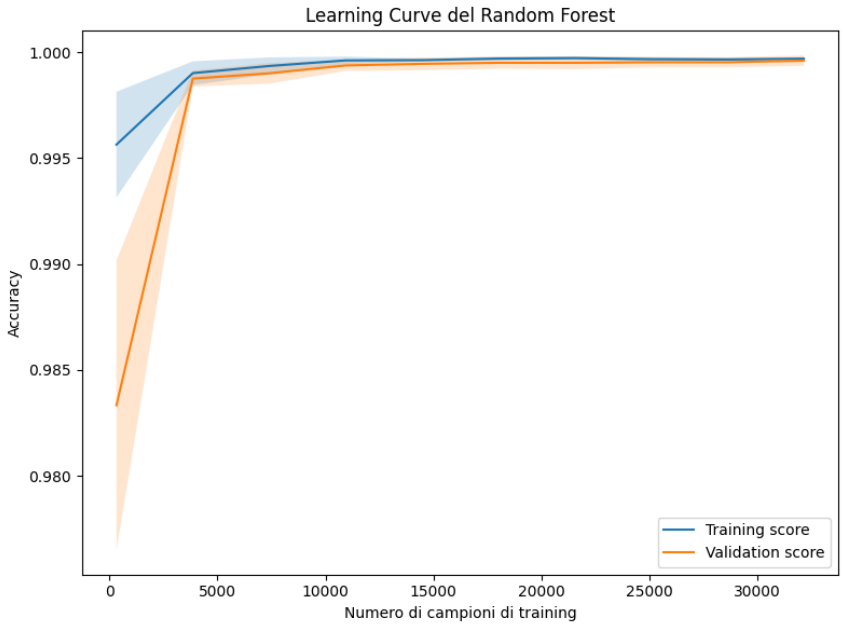
Classe	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	1.00	1.00	1.00	8 778
Picnic	1.00	1.00	1.00	2 184
Accuracy	–	–	1.00	10 962
Macro avg	1.00	1.00	1.00	10 962
Weighted avg	1.00	1.00	1.00	10 962

Tables show that the model is highly performant. The results indicate that the model is in a state of optimal fitting, meaning it generalizes well to the test data while maintaining high accuracy on the training data.

Even from the validation curve, it is clear that there is no overfitting, as evidenced by the absence of any gap between the training and validation scores.

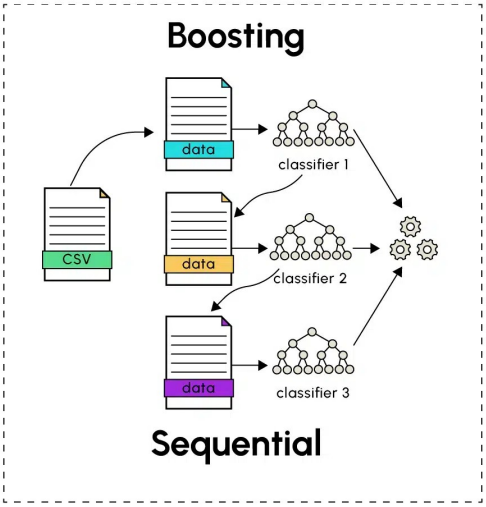
Beyond `max_depth`=3 there is no gain: the curves are perfectly overlapped at 1.00, making it the optimal choice for `max_depth` (we achieve maximum performance and avoiding unnecessary complexity).





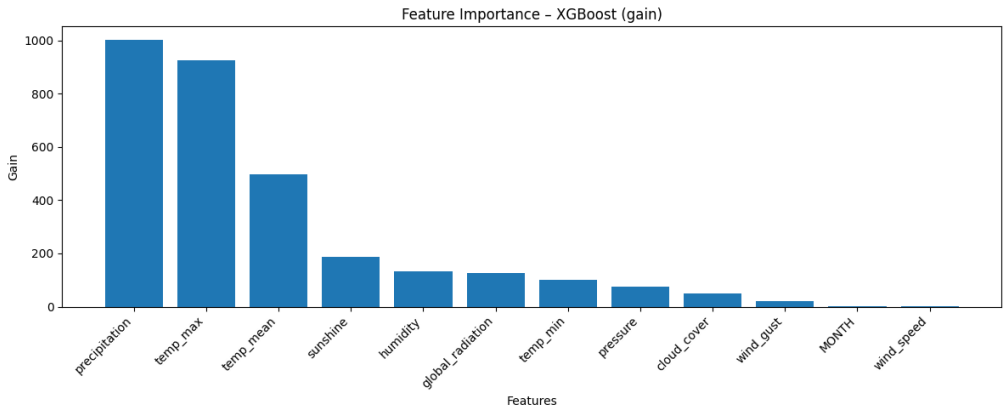
Even with only a few hundred samples, the training accuracy is close to 100%. The small difference between the training and validation scores proves that the ensemble of trees is not "memorizing" noise, but learning robust patterns.

6.3 XGBoost



XGBoost (*eXtreme Gradient Boosting*) is a method that applies **gradient boosting**: it is an ensemble learning technique that builds predictive models by combining many weak decision tree learners, trained in sequence. Each new tree attempts to correct the residual errors of the previous models, thereby reducing the overall error.

As shown in the following plot, the XGBoost model, like the previous Random Forest, considers `_temperature` and `_precipitation` as the most important features (XGBoost gives slightly more weight to `_precipitation`, whereas Random Forest emphasizes `_temperature` more).



As evident from the confusion matrix, the model made 0 errors. In fact, it can be observed that the accuracy and all other considered metrics are at 100%.

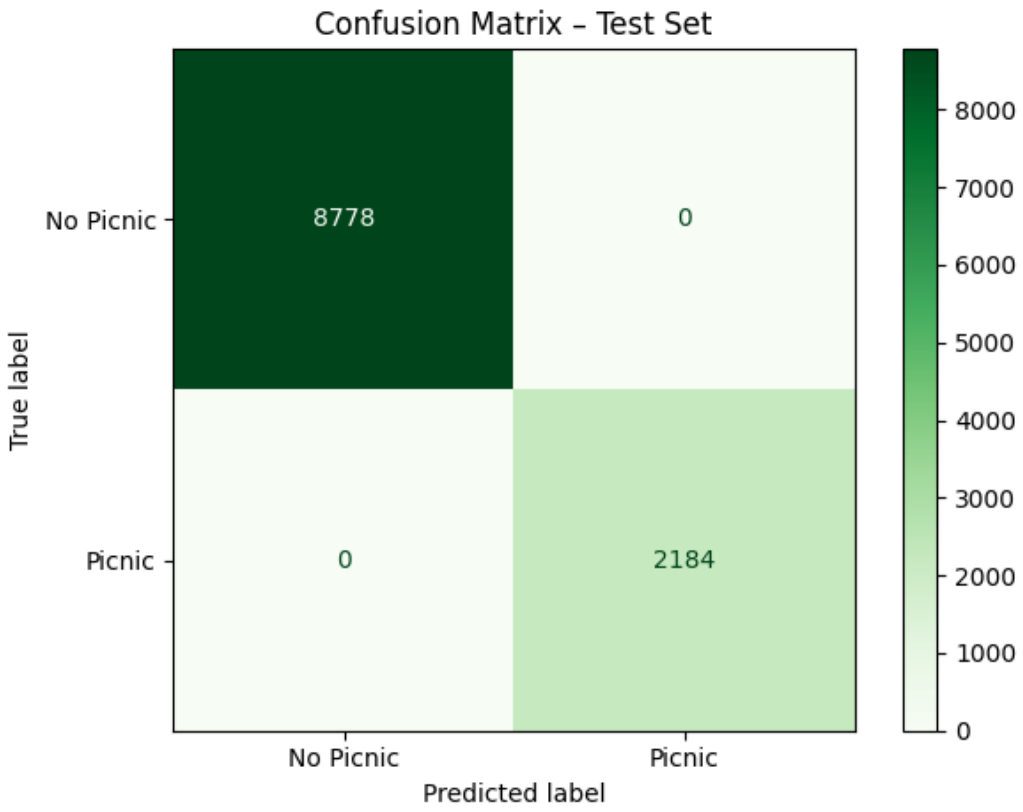


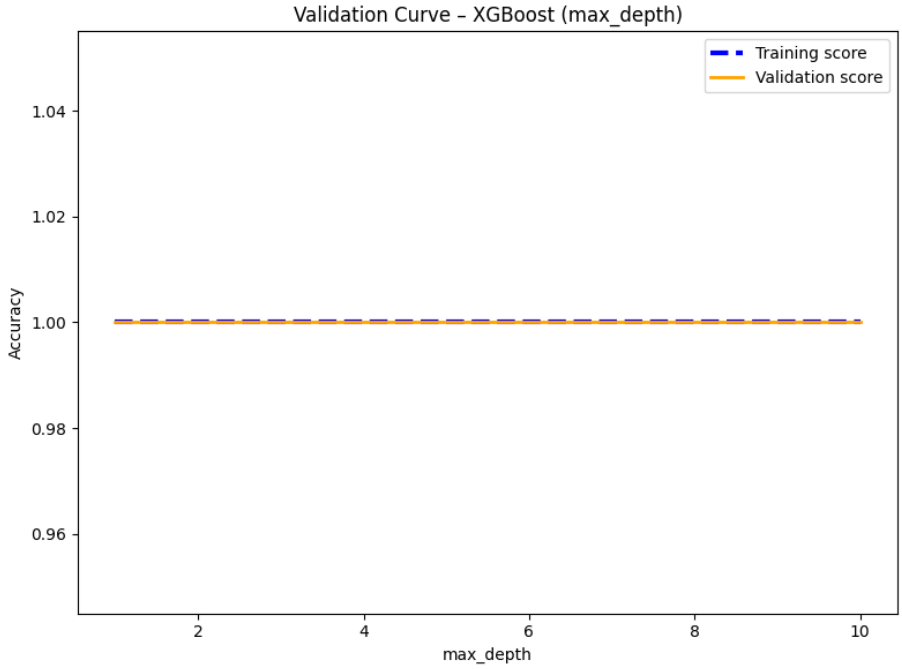
Table 6.5: Classification Report – Training Set (XGBoost)

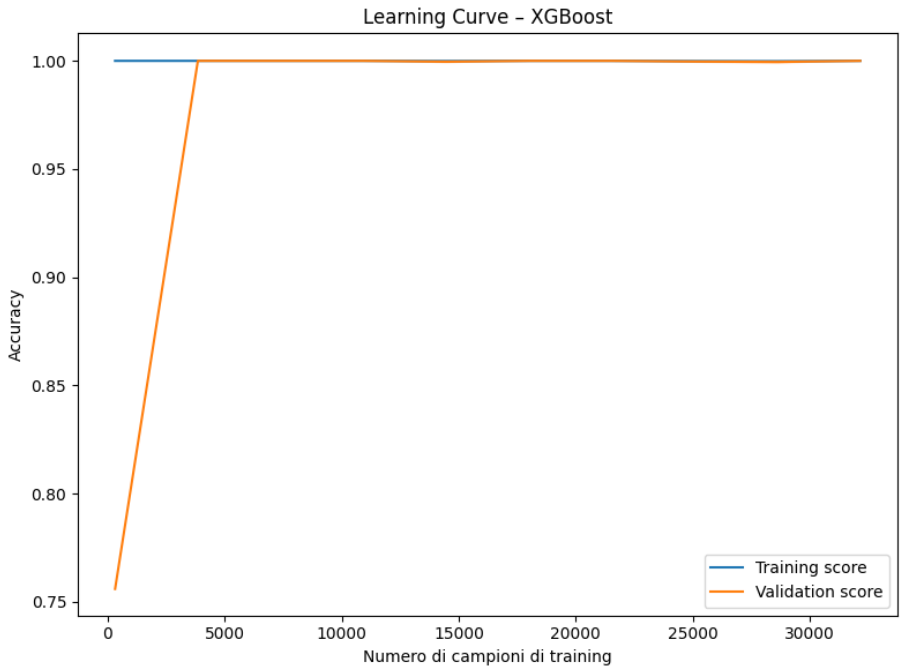
Classe	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	1.00	1.00	1.00	8 778
Picnic	1.00	1.00	1.00	2 184
Accuracy	–	–	1.00	10 962
Macro avg	1.00	1.00	1.00	10 962
Weighted avg	1.00	1.00	1.00	10 962

Table 6.6: Classification Report – Test Set (XGBoost)

Classe	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	1.00	1.00	1.00	8 778
Picnic	1.00	1.00	1.00	2 184
Accuracy	–	–	1.00	10 962
Macro avg	1.00	1.00	1.00	10 962
Weighted avg	1.00	1.00	1.00	10 962

The absence of overfitting can be seen in the validation curve, because there is no gap between the two curves, indicating that the model does not "memorize" the training data relative to the validation data.





From the learning curve, we understand that initially, there was a gap, but with just a few thousand samples the validation curve already aligns perfectly with the training curve.

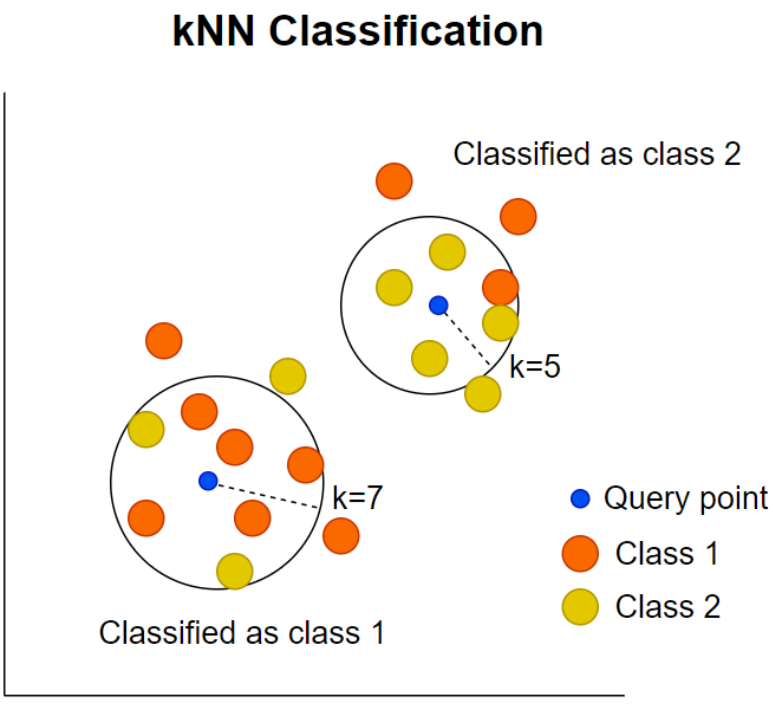
6.4 K-NN

The **k-Nearest Neighbors** (*k-NN*) algorithm is a supervised learning method used for classification tasks, based on the principle that similar objects are close in the feature space. Unlike many other algorithms, k-NN does not build an explicit model during the training phase: it only memorizes the labeled dataset. For a new instance to classify, k-NN computes the distance to all instances in the training set. We will use the Euclidean distance:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

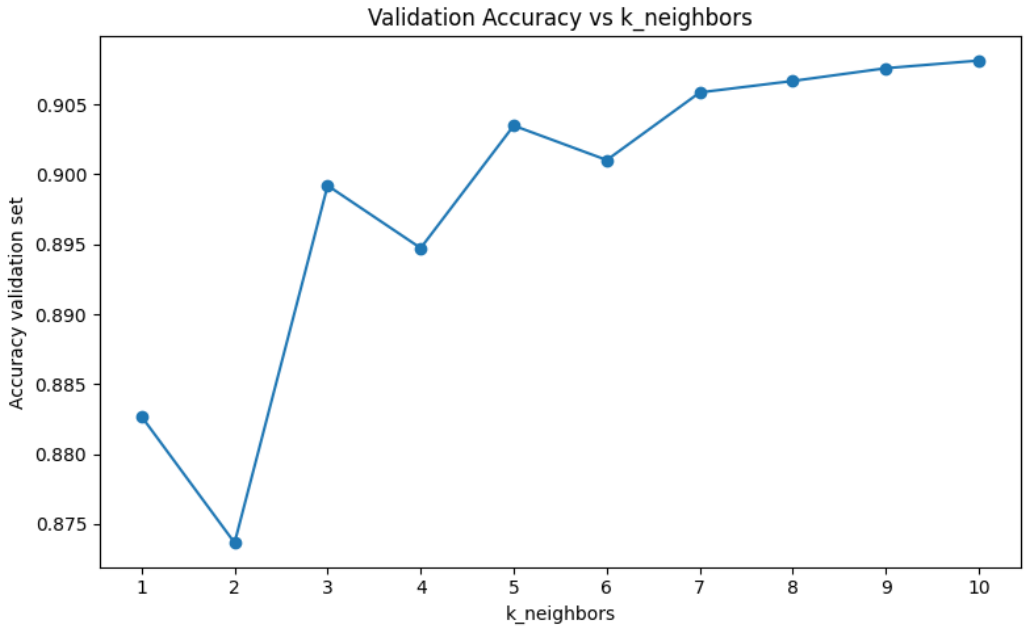
```
"metric": "minkowski",
"p": 2  # Euclidean distance
```

The calculated distances are sorted and the first k instances with the smallest distances are selected. A "majority vote" among the labels of the k neighbors is performed: the instance is assigned to the most frequent class.



To select the k that is right for our data, we run the k-NN algorithm several times with different values of k and choose the k that minimizes the number of errors encountered while maintaining the algorithm's ability to accurately make predictions on unseen data. A too small k may lead to unstable predictions. Conversely, a very large k can lead to increased errors. The k-NN model performs

slightly better for odd values of k and we can observe this from the following graph.



From what has been reported it can be seen that for k = 9 the accuracy of the algorithm is 0.916, the best, as it assigns the labels correctly for a greater number of points.

From the confusion matrix and the classification report, we observe that the k-NN model is not as accurate as the previously tested Random Forest and XGBoost models.

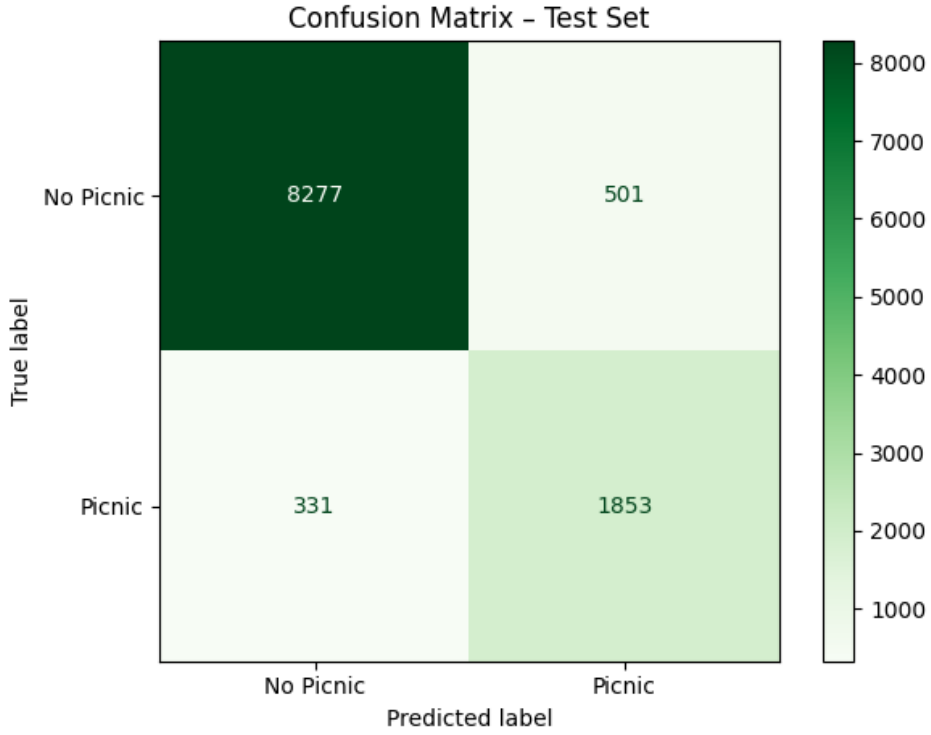


Table 6.7: Classification Report - Training Set (KNN)

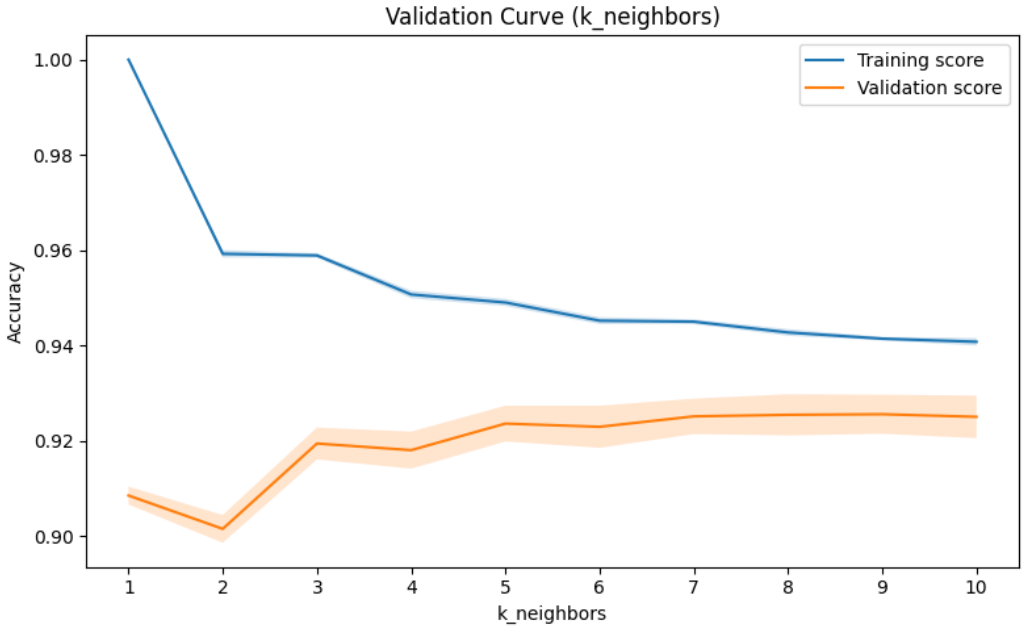
Class	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	0.97	0.95	0.96	30383
Picnic	0.86	0.91	0.89	9811
Accuracy	–	–	0.94	40194
Macro Avg	0.92	0.93	0.92	40194
Weighted Avg	0.94	0.94	0.94	40194

Table 6.8: Classification Report - Test Set (KNN)

Class	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	0.96	0.94	0.95	8778
Picnic	0.79	0.85	0.82	2184
Accuracy	–	–	0.92	10962
Macro Avg	0.87	0.90	0.88	10962
Weighted Avg	0.93	0.92	0.93	10962

k-NN achieves good accuracy on both the training and test sets, showing a slight drop in generalization compared to Random Forest and XGBoost. In particular, the "No Picnic" class is predicted with excellent precision (0.96) and recall (0.94), whereas for "Picnic" precision falls to 0.79 and recall to 0.85, with an F1-score of 0.82. In other words, k-NN performs much better on the majority class.

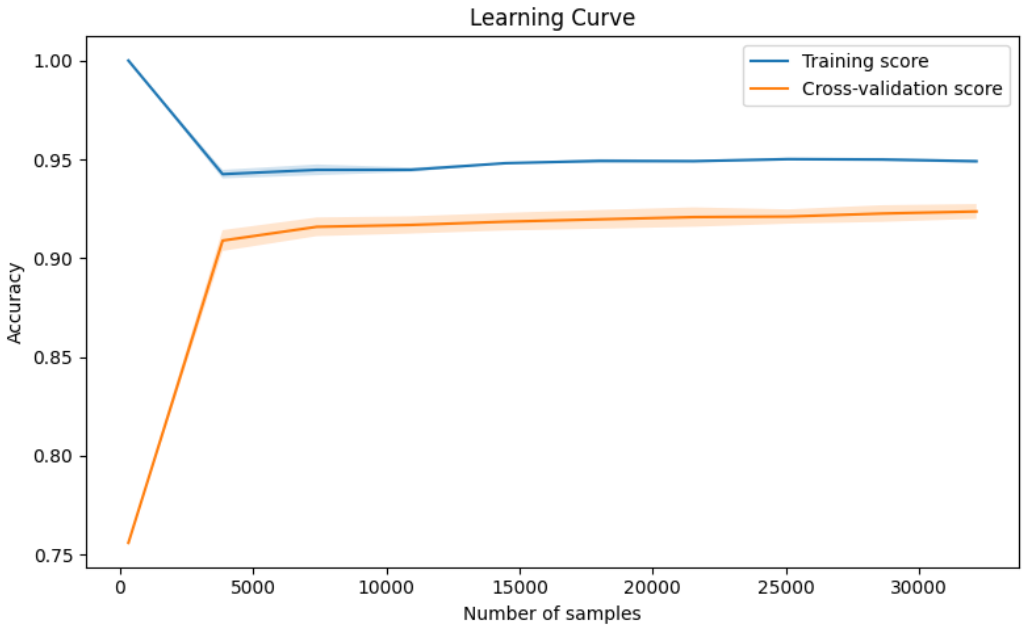
There is a slight sign of **overfitting**: the k-NN model reaches 94% on the training set but falls to 92% on the test set, thus losing about 2 percentage points of accuracy on unseen data. Although this gap is not drastic, it indicates that k-NN is adapting a bit too closely to the training data compared to the validation data. Based on the same considerations made previously, we can immediately conclude that the model does not exhibit optimal fitting.



From the validation curve, we can confirm what was previously observed and also identify the optimal value of k, which, in our case, is 9. Others interesting aspects can be noted:

- as mentioned earlier, the model performs better for odd values of k (as we can see from the zig-zag pattern formed by the two curves);
- we observe that, with low values of k, the model suffers from overfitting. As k increases, performance improves, but the gap between the two curves

never disappears. The training score remains consistently higher than the validation score, indicating slight overfitting.



From the learning curve, we observe that the training score remains consistently above the cross-validation score. Although the gap between them narrows as the number of samples increases, it never fully closes (indicative of slight overfitting).

To solve this overfitting, one could experiment with techniques like

- to balance the classes, such as assigning different class weights, to help the model distinguish the minority class more effectively;
- to experiment with distance metrics other than the default Euclidean.

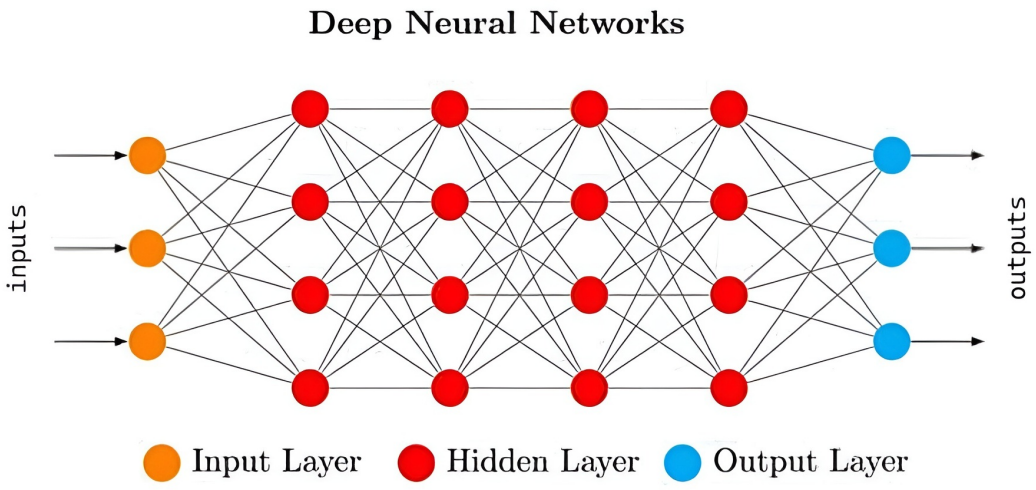
However, we decided to leave the class distribution unchanged to allow a fairer comparison with the other models, for which class balancing wasn't required.

Ultimately, it is more effective to use tree-based ensemble models like those discussed earlier.

6.5 Deep Learning

After applying various classical machine learning methods to solve our classification problem, we now try to use a supervised deep learning approach. **Deep Learning** is a subset of machine learning (ML) that uses neural networks with many layers ("deep") to learn from large amounts of data. Deep learning algorithms attempt to draw similar conclusions as humans would by continually analysing data with a given logical structure. To achieve this, deep learning uses a multilayered structure of algorithms called neural networks.

A **neural network** is a computational model whose structure resembles the network of the neurons in the brain, with layers of connected nodes. It can learn from data, that is, it can be trained to recognize patterns, classify data and predict future events. Neural networks that operate on two or three layers of connected neurons are known as *superficial neural networks*, consisting of an input layer, one or more hidden layers, and an output layer. The layers are interconnected via nodes or neurons, with each layer using the output of the previous layer as input.



In this case, since we need to decide whether to go ahead with the picnic or not, we are faced with a classification problem, in which the algorithm learns to classify new observations based on the assigned labels.

We need to build a neural network; for this purpose we will use **TensorFlow**, an open-source library created by Google that is very useful for classification tasks.

We have used Google Colab to make the code. By default, Colab notebooks run on CPU but, for this task, we had to set the notebook to run on the GPU, change the runtime type, and then select GPU. Why do we need to use a GPU? We use a GPU for several reasons:

- GPU have hundreds and thousands of cores, all of which are dedicated towards a single task;
- GPUs can handle large datasets because they include dedicated video memory (VRAM), faster than RAM memory, allowing the CPU's memory to remain available for other tasks;
- when used in parallel, GPUs can scale more easily than CPUs, enabling much faster processing of very large datasets.

TensorFlow uses GPUs to accelerate model training.

Our neural network structure is composed in this way:

Table 6.9: Neural Network Structure

Layer	**Type**	**Size**	**Activation function**	**Dropout**
Input	Input layer	n_{features}	–	–
Hidden 1	Dense	32	ReLU	0.5
Output	Dense (output)	1	Sigmoid	–

We observe that n_{features} is the number of columns (features) in the preprocessed dataset, i.e. the number of columns remaining in X_train after removing the `DATE`, `picnic_label`, `city`.

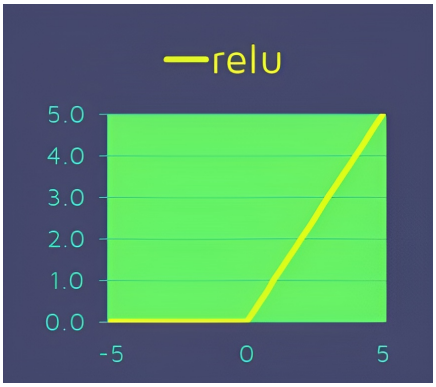
For all of the layers listed above, the size corresponds to the number of neurons present in that particular layer.

A **dense layer**, also known as a fully connected layer, is a fundamental component of neural networks where each neuron is connected to every neuron in the preceding layer.

Dropout is a technique used during the training phase of a neural network to prevent overfitting: it consists on randomly "turns off" some neurons in each training step (*batch*), meaning those neurons are temporarily excluded from the learning process. In our case, dropout equal to 0.5 means that randomly disable 50% of the neurons in that layer at each training step.

As activation function of the hidden layers we chose **ReLU**

$$\text{ReLU}(x) = \max(0, x)$$

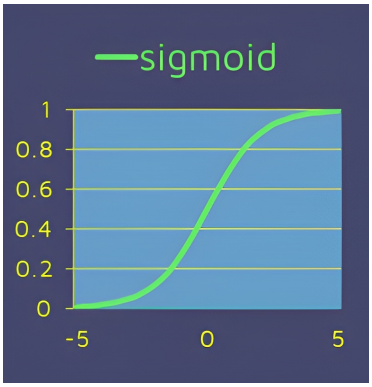


because

- it's extremely simple to calculate (no exponentials or division);
- for negative inputs, reLU outputs 0. This means that many neurons "do not activate" and this make the network more sparsely active, which can improve generalization and reduce overfitting.

As activation function of the output layer we chose **sigmoid**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



because

- the sigmoid function maps any real-valued input to a value between 0 and 1, which can be directly interpreted as the probability that a picnic is possible or not;

- combining sigmoid in the output layer with binary cross-entropy is the standard approach for binary classification.
- compared to more complex functions, the sigmoid is efficient and well-supported.

The chosen hyperparameters are the following.

```
# ------------------------------
# Hyperparameters Neural Network
# ------------------------------
NN_PARAMS = {
    "layers": [32],         # Number of units per layer
    "activation": "relu",       # Activation function
    "dropout_rate": 0.5,        # Dropout between layers
    "learning_rate": 0.001,
    "batch_size": 16,
    "epochs": 100,
    "patience": 40
```

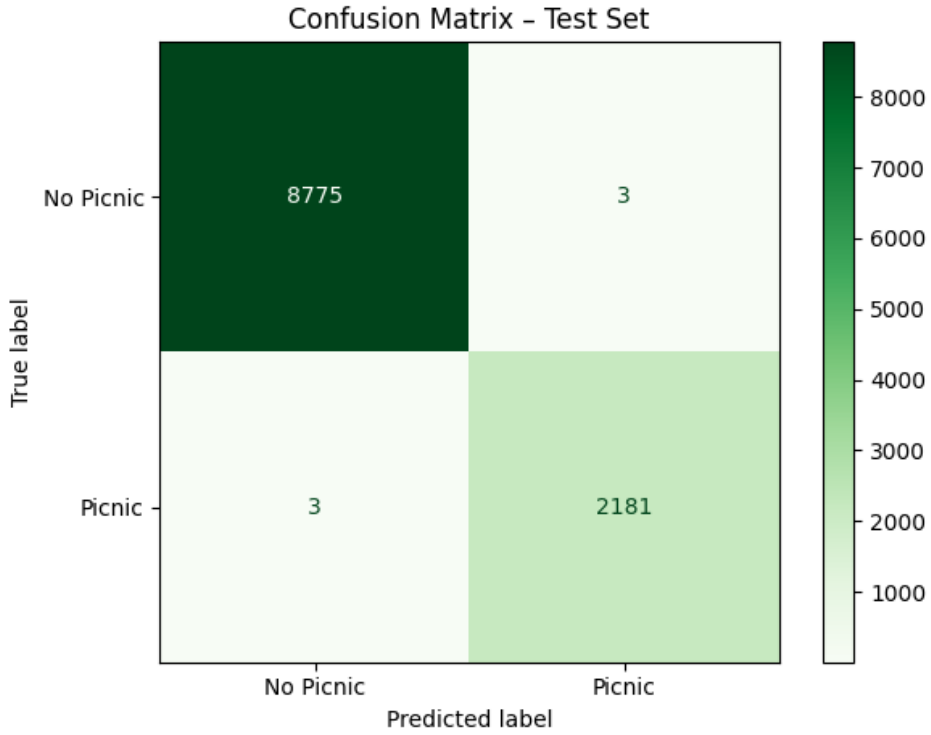
During the training we measured the loss function value and accuracy value on training set and validation set.

We chose as **loss function** the binary cross-entropy. **Binary cross-entropy** (or *log loss*) is the most common cost function for binary classification problems. It measures how far the model's predicted probabilities (the "scores" in $[0, 1]$) deviate from the true labels (0 or 1).

$$\ell(y, \hat{y}) = -\Big[y \, \log(\hat{y}) + (1 - y) \, \log(1 - \hat{y})\Big]$$

- $y \in \{0, 1\}$ is the true label,
- $\hat{y} \in (0, 1)$ is the estimated probability for class "1" (the output of the sigmoid).

The confusion matrix shows excellent performance.



From the tables below, we see that the model has a perfect fit.

Table 6.10: Classification Report: TRAIN SET

Class	**Precision**	**Recall**	**F1-score**	**Support**
No picnic	1.00	1.00	1.00	30383
picnic	1.00	1.00	1.00	9811
Accuracy	–	–	1.00	40194
Macro Avg	1.00	1.00	1.00	40194
Weighted Avg	1.00	1.00	1.00	40194

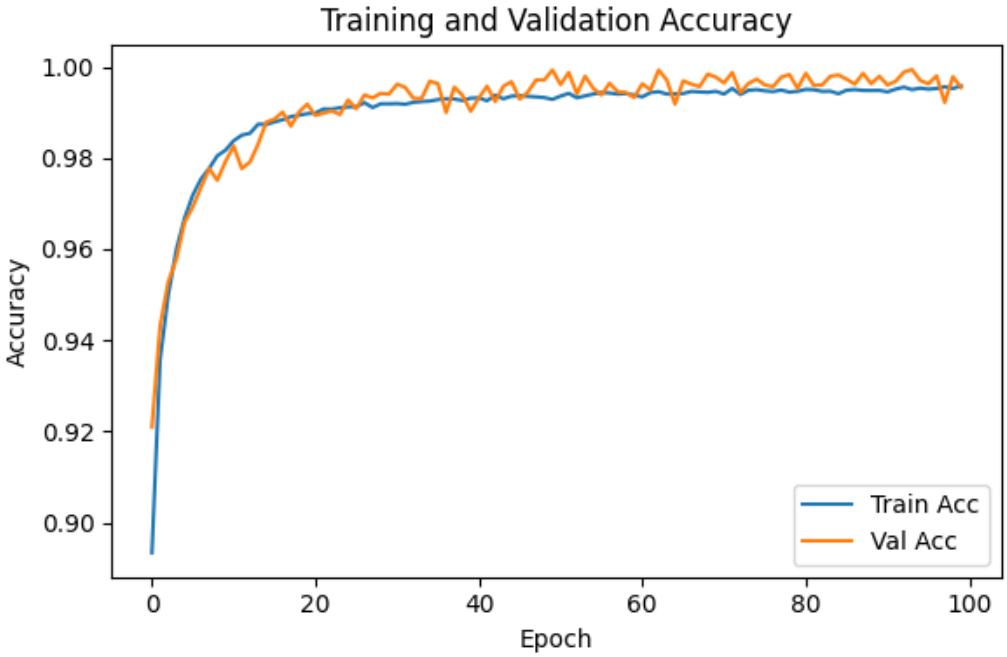
Table 6.11: Classification Report: TEST SET

Class	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	1.00	1.00	1.00	8778
Picnic	1.00	1.00	1.00	2184
Accuracy	–	–	1.00	10962
Macro Avg	1.00	1.00	1.00	10962
Weighted Avg	1.00	1.00	1.00	10962

The following two graphs confirm what we observed from the classification reports: indeed, we notice that the two curves are very close to each other and occasionally overlap.

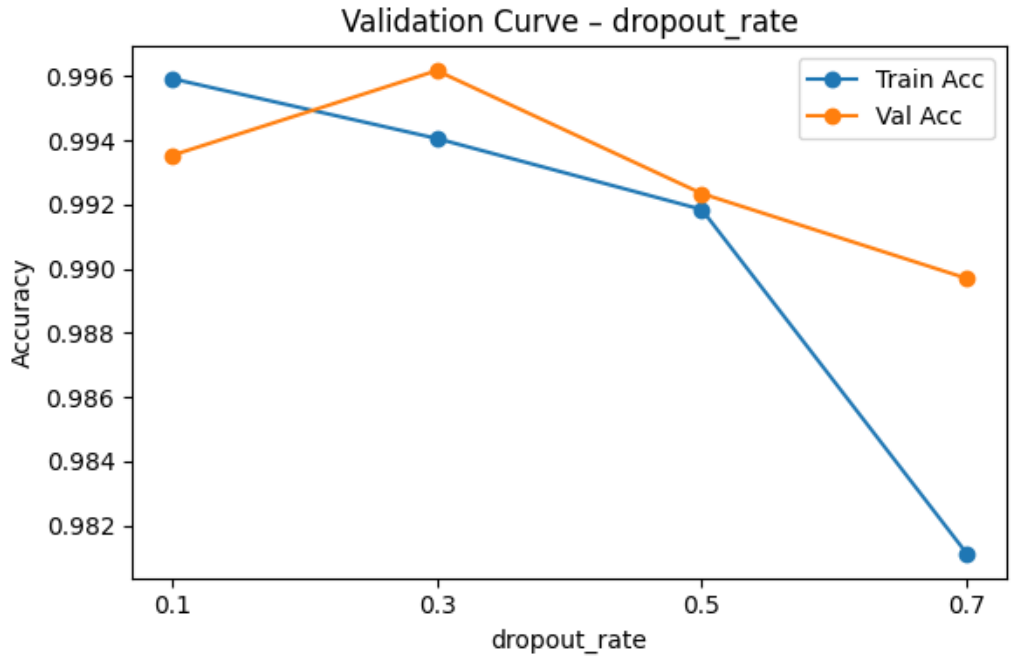


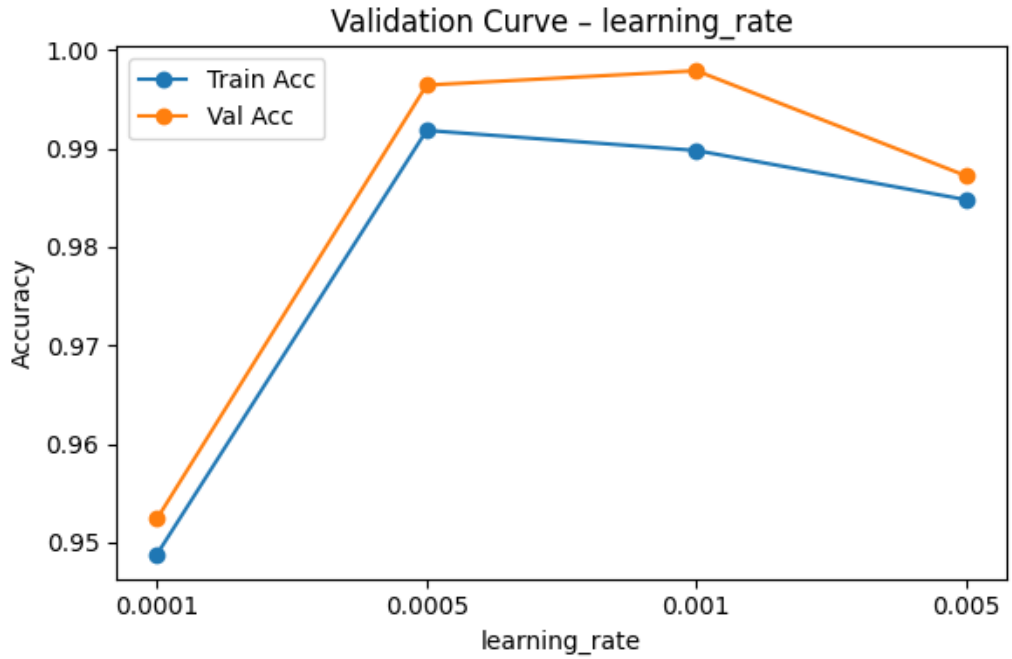
The fact that the validation loss always remains slightly below the training loss implies that the choice of parameters effectively counteracts overfitting.

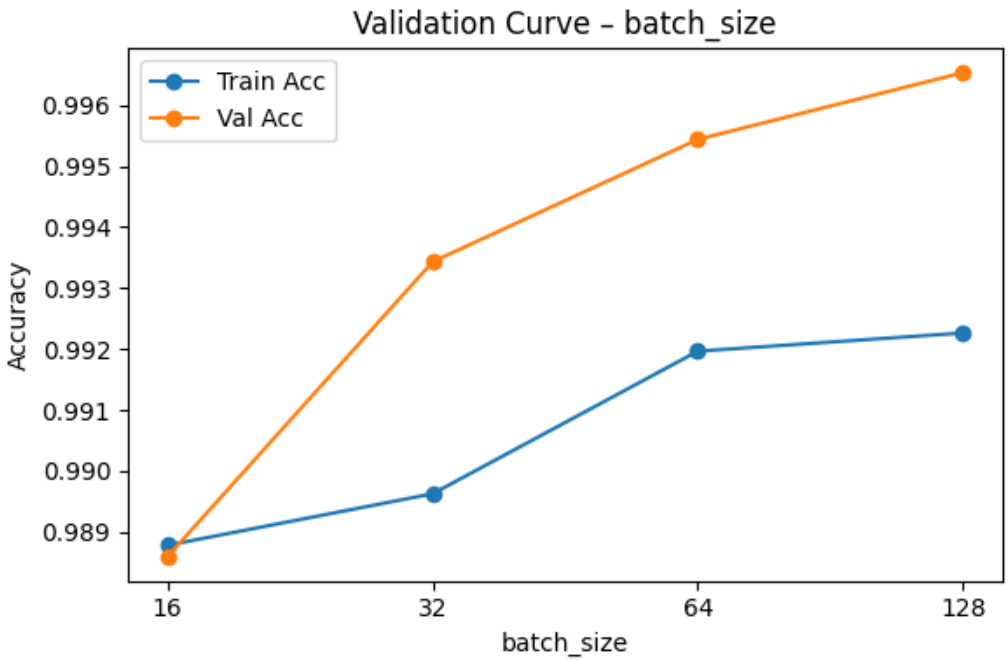


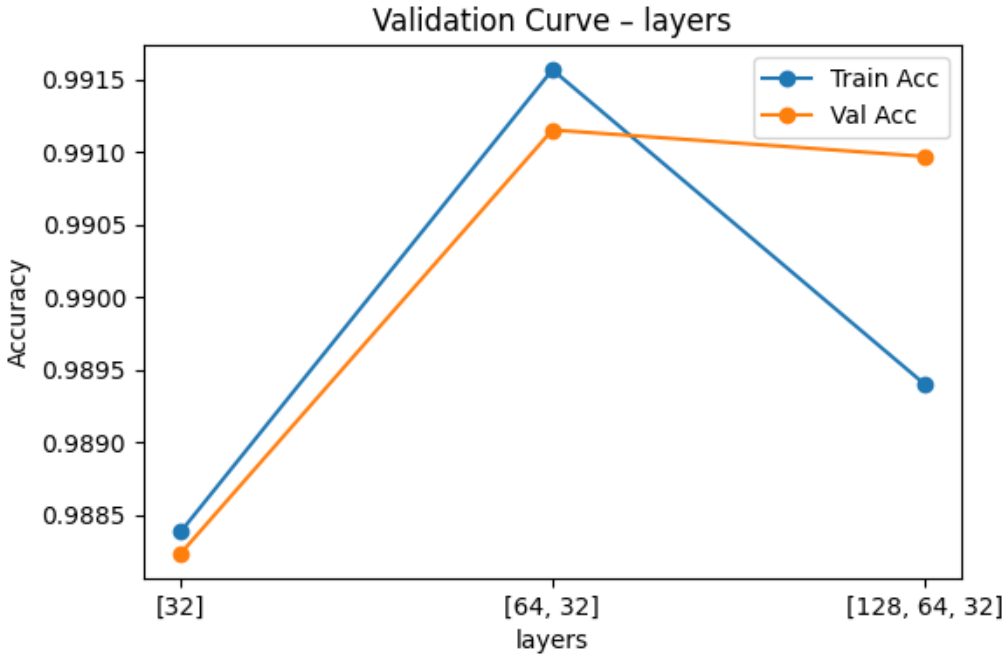
The training and validation accuracy curves climb steeply from around 0.88 at epoch 0 to above 0.98 by about epoch 15. This indicates the network quickly learns. After convergence, both curves settle around 0.99 (i.e. 99%). This suggests the model has captured the relationship between features and labels extremely well. The *fluctuations* remain within a tight band, reflecting a robust model (no overfitting).

All the following validation curves provide the best values for the chosen hyperparameters, thus justifying their selection.





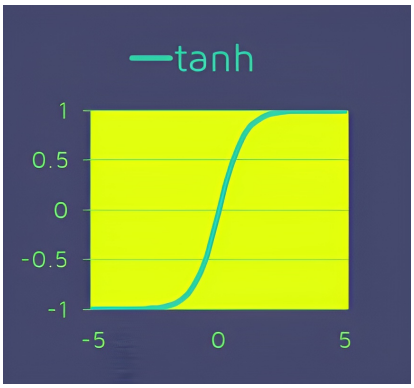

Validation Curve – batch_size
Train Acc
Val Acc
Accuracy
0.996
0.995
0.994
0.993
0.992
0.991
0.990
0.989
16
32
64
128
batch_size


Validation Curve – layers
Train Acc
Val Acc
Accuracy
0.9915
0.9910
0.9905
0.9900
0.9895
0.9890
0.9885
[32]
[64, 32]
[128, 64, 32]
layers

We also tried using tanh as the activation function, but the results did not differ much from those obtained with ReLU, even though ReLU, unlike tanh, is not zero-centered. This happens because whenever the dataset contains a negative value, ReLU always maps it to zero (the so-called "dead neuron"), and in many situations where negative values carry significant information, this causes information loss and poorer model generalization. In contrast, tanh can represent negative values as well, which yields better accuracy.

The **tanh** function is defined as:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Alternatively, it can also be written using the sigmoid function $\sigma(x)$:

$$\tanh(x) = 2 \cdot \sigma(2x) - 1 \quad \text{where} \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

Unlike the sigmoid function (which outputs values in $[0, 1]$), `tanh` produces values in the range $[-1, +1]$, centered around 0. This helps make training more stable and leads to faster convergence.

It is slightly more expensive computationally than ReLU.

Function	**Output Range**	**Zero-centered**	**Sparsity**	**Speed**
ReLU	$[0, +\infty)$	No	High	Fast
tanh	$[-1, +1]$	Yes	Low	Medium

The chosen hyperparameters are the following.

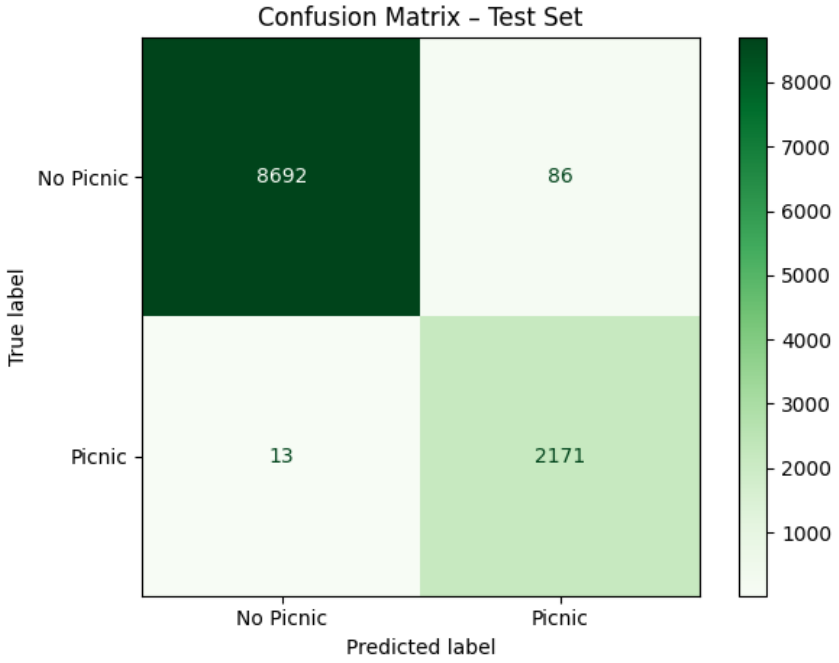
```
# ----------------------------
# Hyperparameters Neural Network
# ----------------------------
NN_PARAMS = {
    "layers": [64, 32],         # Number of units per layer
    "activation": "tanh",       # Activation function
    "dropout_rate": 0.3,        # Dropout between layers
    "learning_rate": 0.0001,
    "batch_size": 64,
    "epochs": 100,
    "patience": 40
```

Our neural network structure is composed in this way:

Table 6.12: Neural Network Structure

Layer	**Type**	**Size**	**Activation function**	**Dropout**
Input	Input layer	n_{features}	–	–
Hidden 1	Dense	64	Tanh	0.3
Hidden 2	Dense	32	Tanh	0.3
Output	Dense (output)	1	Sigmoid	–

The following confusion matrix highlights the good performance of the neural network (with tanh as the activate function).



From the classification reports we immediately notice that, this time too, the neural network seems to adapt perfectly.

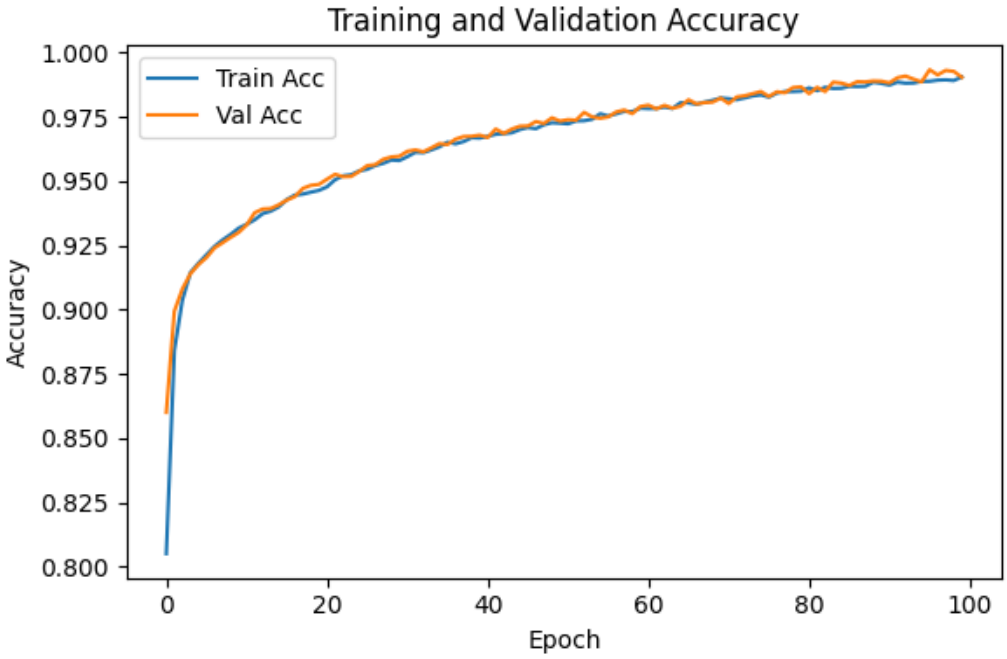
Table 6.13: Classification Report: TRAIN SET

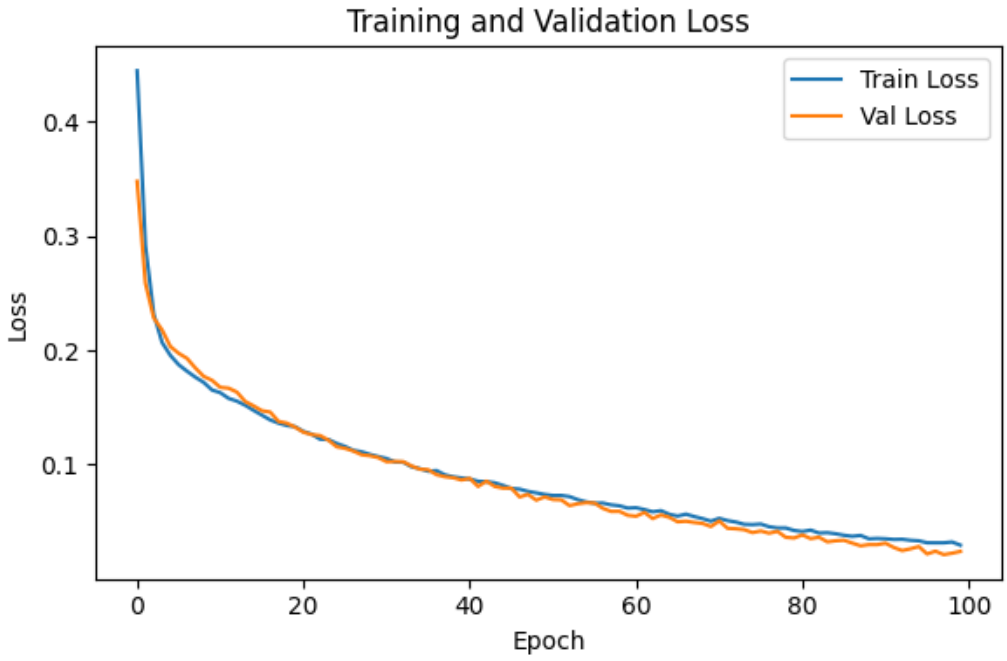
Class	**Precision**	**Recall**	**F1-score**	**Support**
No picnic	1.00	0.99	1.00	30383
picnic	0.98	1.00	0.99	9811
Accuracy	–	–	0.99	40194
Macro Avg	0.99	1.00	0.99	40194
Weighted Avg	0.99	0.99	0.99	40194

Table 6.14: Classification Report: TEST SET

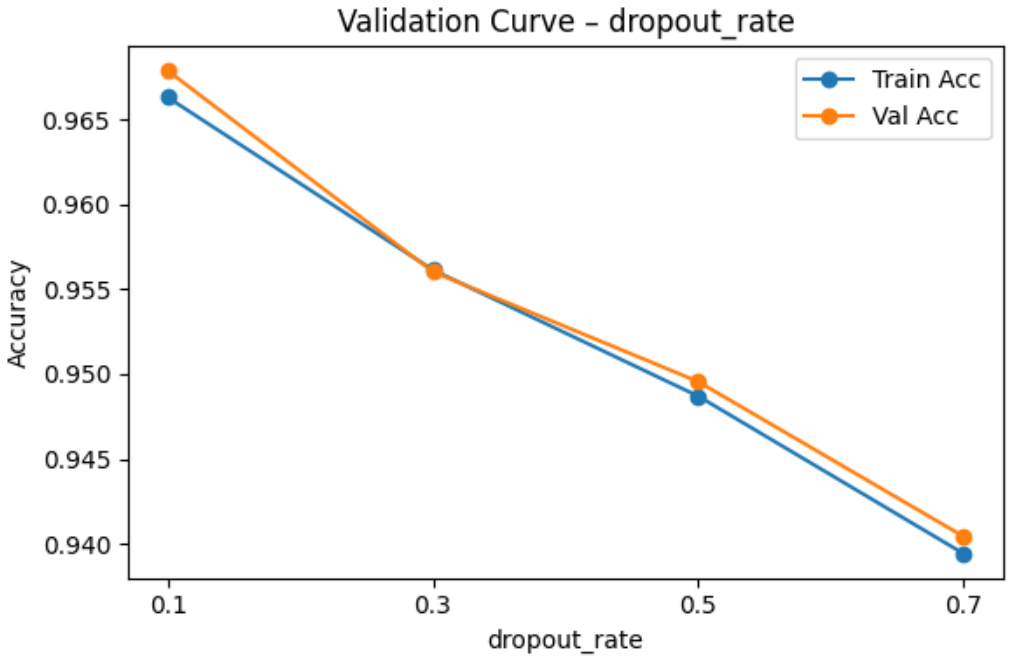
Class	**Precision**	**Recall**	**F1-score**	**Support**
No Picnic	1.00	0.99	0.99	8778
Picnic	0.96	0.99	0.98	2184
Accuracy	–	–	0.99	10962
Macro Avg	0.98	0.99	0.99	10962
Weighted Avg	0.99	0.99	0.99	10962

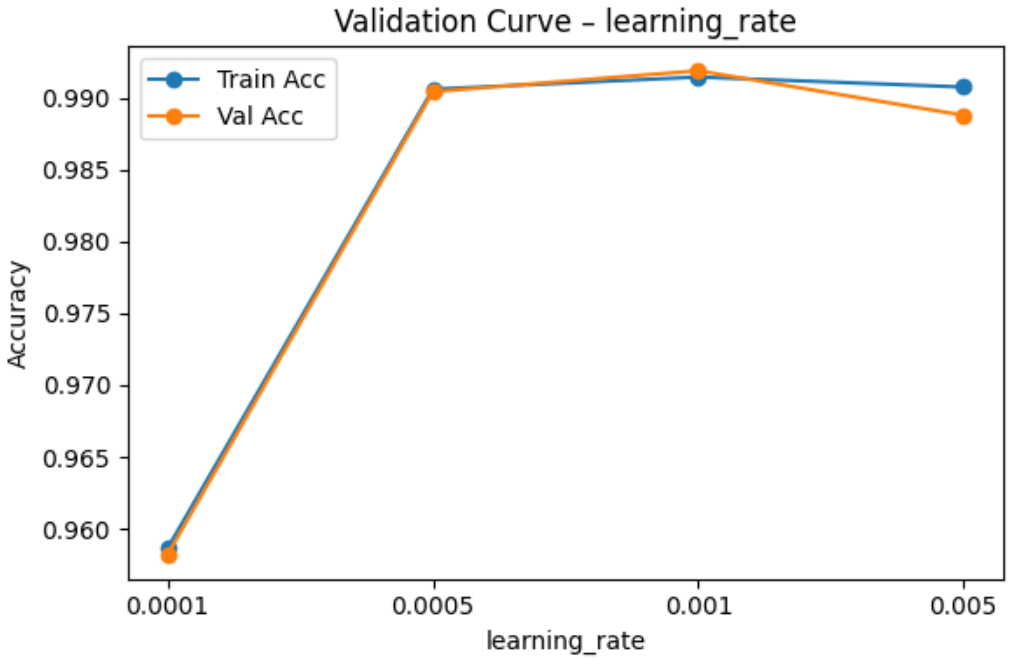
The two following graphs show that both the training and validation curves follow the same trend, overlapping almost perfectly (a sign of perfect fit).

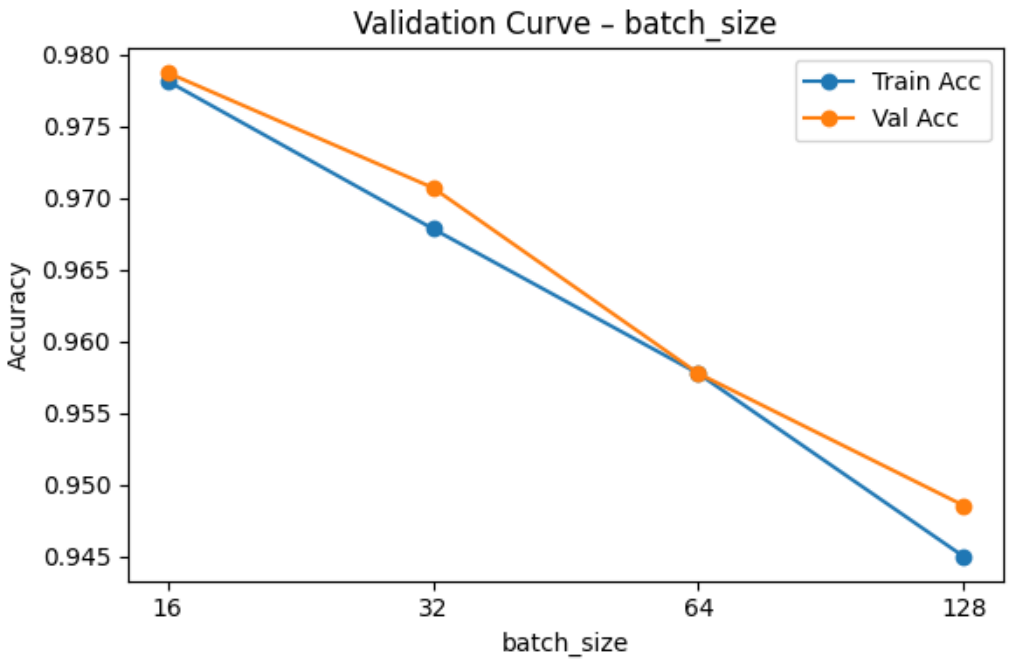


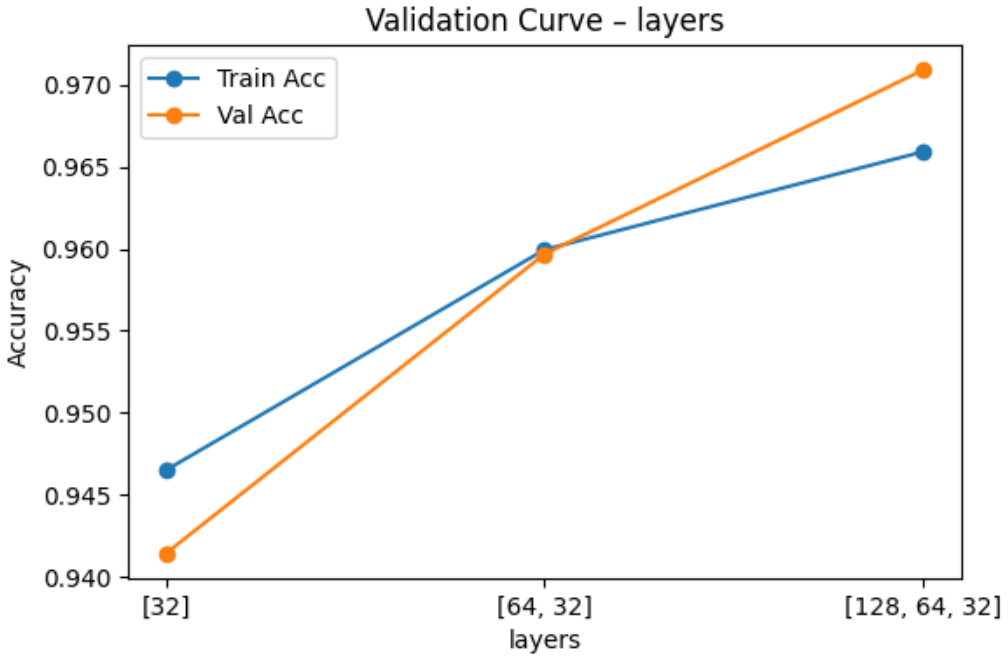


All the following validation curves provide the best values for the chosen hyperparameters, thus justifying their selection.






Validation Curve – batch_size
Train Acc
Val Acc
Accuracy
0.980
0.975
0.970
0.965
0.960
0.955
0.950
0.945
16
32
64
128
batch_size


Validation Curve – layers
Train Acc
Val Acc
Accuracy
0.970
0.965
0.960
0.955
0.950
0.945
0.940
[32]
[64, 32]
[128, 64, 32]
layers

Chapter 7

Conclusion

In this analysis, we compared five classification approaches for predicting weather conditions suitable for a picnic: Decision Tree, Random Forest, XGBoost, K-Nearest Neighbors (K-NN), and Neural Network (Deep Learning). Each method was evaluated in terms of accuracy, robustness to overfitting, and interpretability, using separate test and validation datasets to ensure generalizability.

Table 7.1: Model performance comparison (Test Set)

Model	**Accuracy**	**Training Time (*s*)**	**Interpretability**
Decision Tree	0.90	0.025	**High**
Random Forest	1.00	1.632	Medium
XGBoost	1.00	0.377	Medium
k-NN	0.92	0.082	Low
Neural Network (reLU)	0.99	846.398	Low
Neural Network (tanh)	0.99	217.108	Low

Decision Tree showed good performance (around 90% accuracy on both training and test sets) with a model that is easily interpretable through the calculation of Gini Impurity and the learning and validation curves. By limiting the maximum depth to three levels (`max_depth=3`), the model avoided both overfitting and underfitting, while still remaining sensitive to local variations in the dataset.

Random Forest e XGBoost performed equally in terms of accuracy, achieving 100% on the training and test sets. Both ensemble methods confirmed

the predominant importance of variables related to temperature and precipitation, and showed a complete absence of errors in the confusion matrix, thanks to the use of multiple uncorrelated trees and the iterative boosting of residuals.

K-NN with $k = 9$, it achieved slightly lower accuracy (around 92%), showing a small gap between training and test performance, indicating a mild tendency toward overfitting. The metrics remained satisfactory for the majority class (*no picnic*), while the minority class (*picnic*) showed more limited performance.

Neural Networks delivered excellent results, with accuracy 99% and overlapping learning curves, both with the ReLU and tanh activation functions. The neural networks demonstrated strong generalization capabilities, although it requires higher computational resources and presents greater complexity in interpreting weights and activations.

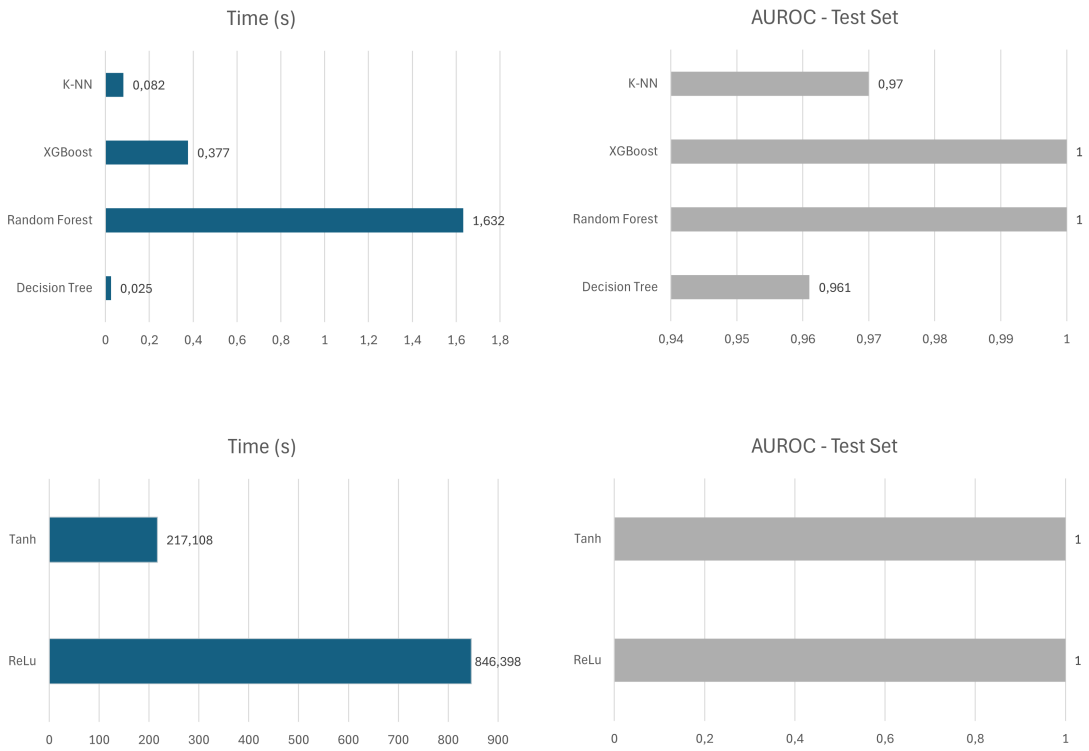
To evaluate model performance on the test set, we used the **AUROC** metric: it measures the ability of a classifier to distinguish between the two target classes across all possible classification thresholds. A score of 1.0 indicates perfect discrimination, while a score of 0.5 corresponds to random guessing. This metric is particularly useful in imbalanced classification problems, as it is insensitive to class distribution.

As shown in the figure below, the Decision Tree is by far the fastest, with a training time of 0.025s, yet it achieves an AUROC of 0.961. K-NN requires 0.082s for training, obtaining a slightly higher AUROC (0.970). XGBoost and Random Forest both reach a perfect AUROC (1.000), but at the expense of longer training times: 0.377s for XGBoost and 1.632s for Random Forest.

Regarding the comparison of activation functions in a neural network model, the Tanh function proved to be much more efficient than ReLU: training with Tanh took 217s versus 846s with ReLU. Moreover, Tanh achieved an AUROC of 1.000, while ReLU peaked at 0.999. Therefore, in the experimental setup considered, Tanh outperformed ReLU in both training speed and test-set accuracy.

In conclusion, for applications that demand the highest accuracy without strict time constraints, ensemble algorithms (XGBoost, Random Forest) and the Tanh activation function are the preferred choices. If very fast responses

are required, Decision Tree or K-NN provide valid solutions with an acceptable trade-off in performance.



Considering the balance between performance, interpretability, and computational complexity, tree-based ensembles (Random Forest and XGBoost) emerge as the optimal choice. They combine high accuracy, robustness to overfitting, and low training time, while also providing a level of interpretability through feature importance analysis that supports practical decision-making.

The high accuracy achieved by ensemble methods suggests that picnic suitability prediction is a *highly tractable problem* with proper feature engineering, outperforming traditional meteorological heuristics.